Low Level System Design, Design Patterns & SOLID Principles

1. OOP Recap
2. SOLID Principles
3. Design Patterns
    a. Behavioural
    b. Creational
    c. Structural

Module 1: Object Oriented Programming Recap

Class & Object

- **Class** → Blueprint/template (defines properties + behavior)
- **Object** → Instance of a class (real entity)
    - Example: Car (class), BMW (object)

Encapsulation

- Binding data (variables) and methods (functions) together into a single unit, usually a class.
- Control **access to data** via access modifiers (e.g., private, public).
    - Example: **Getters and setters** in a class to access and update private data members.

Abstraction

- Hiding unnecessary details and showing only the essential information.
- Focus on **what an object does**, not **how it does it**.
    - Example: Defining a method draw() in a Shape class without revealing the drawing logic (implemented by subclasses like Circle, Rectangle).

Inheritance

- Mechanism for creating new classes based on existing classes.
- Helps in code reuse and establishing relationships between objects.
- Example: Car class inherits from Vehicle class and adds specific behavior like startEngine().

Polymorphism

- Allows one interface to be used for a general class of actions.

- Achieved through method overriding (runtime polymorphism) or method overloading (compile-time polymorphism).
    - Example: A single method area() can be used for different objects like Circle, Rectangle, with different implementations.

Design a Payment Service

- For this example, we can assume payment service should be responsible for storing details of Payments Methods (Credit Card, Debit Card etc) that belong to a user and also have a makePayment() Method.

Object-Oriented Programming (OOP) - Payment System Design

This document explains the implementation of a flexible payment system using core OOP principles including **Inheritance**, **Abstraction**, **Interface Implementation**, and **Polymorphism**.

Design Problem & Solution

Step 1: Initial Problem - Code Duplication

Problem: Creating separate classes for CreditCard and DebitCard resulted in duplicate code for common attributes.

Solution: Create a parent `Card` class to hold shared attributes.

```
// Parent Class - Card.java
public abstract class Card {
    private String cardNumber;
    private String cardHolderName;

    public Card(String cardNumber, String cardHolderName) {
        this.cardNumber = cardNumber;
        this.cardHolderName = cardHolderName;
    }
}
```

Benefit:

- ✅ Eliminates code duplication
- ✅ Single source of truth for card attributes
- ✅ Easier maintenance and updates

Step 2: Adding Abstract Methods

Problem: Different card types (CreditCard, DebitCard) need different payment implementations.

Solution: Define an abstract `pay()` method in the parent class and implement it differently in child classes.

```
// Child Classes implementing pay() differently
public class CreditCard extends Card {
    @Override
    public void pay() {
        System.out.println("Payment made using Credit Card\n...");
    }
}

public class DebitCard extends Card {
    @Override
    public void pay() {
        System.out.println("Payment made using Debit Card\n...");
    }
}
```

Benefits:

- ✅ Enforces specific behavior in child classes
- ✅ Maintains contract through abstraction
- ✅ Supports **method overriding** for polymorphic behavior

Step 3: Extending Beyond Cards - Interface Implementation

Problem: We need to support payment methods like UPI and Wallet that are NOT cards.

Solution: Create a `PaymentMethod` interface to define a contract for all payment types.

```
// Interface - PaymentMethod.java
public interface PaymentMethod {
    void pay();
}
```

Now, both card and non-card payment methods implement this interface:

```
// Card classes now implement the interface
public abstract class Card implements PaymentMethod { ... }

// Non-card payment methods
public class UPI implements PaymentMethod { ... }
public class Wallet implements PaymentMethod { ... }
```

Benefits:

- ✅ Supports multiple payment types without inheritance hierarchy
- ✅ Loose coupling - different implementations of same contract
- ✅ Single Responsibility Principle - each class handles one payment type

Step 4: Unified Payment Processing - Polymorphism

Problem: Client needs a single way to process different payment types.

Solution: Use `PaymentService` with **Runtime Polymorphism** to handle all payment methods uniformly.

```java
// PaymentService.java - Uses Polymorphism
public class PaymentService {
    HashMap<String, PaymentMethod> paymentMethods;

    public void addPaymentMethod(String name, PaymentMethod payMethod) {
        paymentMethods.put(name, payMethod);
    }

    public void makePayment(String name) {
        PaymentMethod method = paymentMethods.get(name);
        if (method != null) {
            method.pay(); // 💡 Runtime Polymorphism in action!
        }
    }
}
```

How it works:

- At runtime, the JVM determines which `pay()` method to call based on the actual object type
- Same interface, different behavior for each payment method
- No casting or type checking needed

Benefits:

- ✅ Clean, maintainable code ✅ Easy to extend with new payment methods ✅ Runtime flexibility

---

Class Structure Diagram

```
┌──────────────────────────┐
│ PaymentMethod      │ (Interface)
│ + pay()            │
└──────────────────────────┘
      ▲
      │ implements
      │
   ┌──┴──────────┬──────────────┬──────────────┬──────────────┐
   │             │              │              │
   │             │              │              │
┌──────────────┐ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│  Card    │ │  UPI    │ │ Wallet   │ │ Other   │
│(Abstract) │ │ (Concrete)│ │(Concrete)│ │ Methods  │
└──────────────┘ └──────────────┘ └──────────────┘ └──────────────┘
     │
     │ extends
     │
   ┌──┴──────────┬──────────┐
   │             │
┌──────────────┐ ┌──────────────┐
│CreditCard│ │DebitCard │
│(Concrete)│ │(Concrete)│
└──────────────┘ └──────────────┘
```

Key OOP Principles Applied

Principle	Implementation	Benefit
Encapsulation	Private attributes with getters/setters in `Card`	Data protection & controlled access
Abstraction	Abstract `Card` class & `PaymentMethod` interface	Hide implementation details
Inheritance	`CreditCard` and `DebitCard` extend `Card`	Code reuse, common behavior
Polymorphism	`PaymentService` handles all `PaymentMethod` types	Flexible, extensible design
Single Responsibility	Each class handles one payment type	Maintainability
Open/Closed Principle	Open for extension (new payment methods), closed for modification	Easy to add new payment types

---

Real-World Example - Usage

```
// Client.java - How to use the system
PaymentService service = new PaymentService();

// Register multiple payment methods
service.addPaymentMethod("gv-CreditCard", new CreditCard("1234", "Chirag"));
service.addPaymentMethod("gv-DebitCard", new DebitCard("5678", "Deepak"));
service.addPaymentMethod("gv-UPI", new UPI("udapi@0011"));
service.addPaymentMethod("gv-wallet", new Wallet("wallet@00w12"));

// Process payments using polymorphism
service.makePayment("gv-CreditCard");  // Calls CreditCard.pay()
service.makePayment("gv-UPI");         // Calls UPI.pay()
service.makePayment("gv-wallet");      // Calls Wallet.pay()
```

Extensibility - Adding New Payment Methods

To add a new payment method like **PhonePe**, simply:

No modifications needed to existing classes! This is the power of good OOP design.

Summary

This payment system demonstrates how OOP principles create:

- ✅ **DRY Code** - No duplication through inheritance
- ✅ **Flexible Design** - Easy to extend with new payment methods
- ✅ **Clean Code** - Clear separation of concerns
- ✅ **Maintainable System** - Changes isolated to specific classes
- ✅ **Real-world Applicability** - Mimics actual payment processing systems

---

UML - Unified Modeling Language

UML Diagrams

- Unified Modeling Language to model systems

- The idea is to have a uniform way to represent the classes, objects, relationships and interactions within simple or complex systems to make it easier for developers and stakeholders to understand and communicate about the system.

UML Diagrams Importance

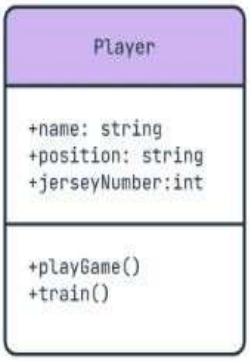
- **Visualization**: UML diagrams provide a visual representation of a system, making it easier to understand the structure, relationships, and interactions between components.
- **Documentation**: They serve as detailed documentation for the software architecture, which is useful for maintaining and scaling the system.
- **Communication**: UML diagrams are a common language for software designers, developers, and stakeholders, facilitating communication about the design.
- **Standardization**: Since UML is a standardized language, it ensures that everyone involved understands the system's design in the same way.

Basic Elements

1. **Class**: A blueprint for objects, defined with a name, attributes, and methods.
   - Example: class User { name, age, login() }
2. **Interface**: A contract that defines methods that a class must implement.
   - Example: interface Loginable { login() }
3. **Object**: An instance of a class at runtime.
4. **Association**: A relationship between two classes that represents interactions between objects.
5. **Inheritance**: Represents an "is-a" relationship, where a subclass inherits from a superclass.
6. **Composition**: A stronger association where one object is part of another and cannot exist independently.
7. **Aggregation**: A weaker form of association where one object contains another, but they can exist independently.

Class Diagrams

Class Diagram: Represents the static structure of a system, showing classes, attributes, methods, and the relationships between them (inheritance, association, etc.).



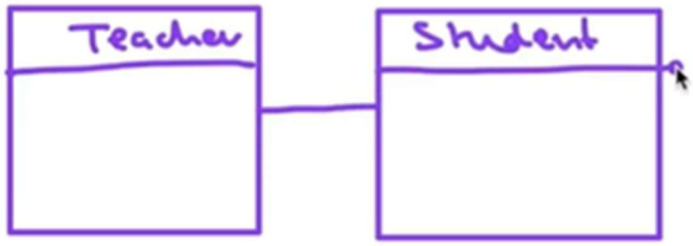
Association

Association represents a relationship between two or more classes. In this case, each object in one class is associated with one or more objects of another class.

Association: Two classes are related, but independent

Understanding: Teacher uses Student Both can exist independently

Real-world: Teacher ↔ Student



Aggregation

Aggregation is a weak "has-a" relationship where one class contains objects of another class. However, the contained objects can exist independently of the container object.

Meaning: One class has another, but both can exist separately

In this example, a **Department** is composed of multiple **Professors**, but the professors exist independently. Even if the department is dissolved, the professors can still exist.

Composition

Composition is a strong **"has-a"** relationship, where one class owns objects of another class. If the container object is destroyed, the contained objects are destroyed as well.

One class fully owns another (lifecycle dependent)

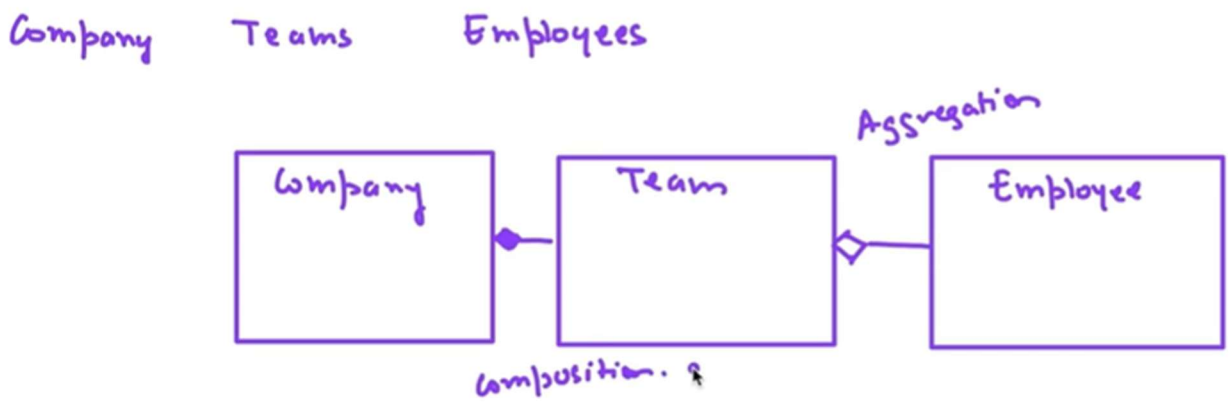
Composition → “strong HAS-A”

```
class House { private List<Room> rooms = new ArrayList<>(); }

class Room { }
```

Understanding: House HAS-A Rooms If house is destroyed → rooms are also destroyed

Real-world: House → Rooms



Inheritance

Inheritance defines an **"is-a"** relationship where a subclass inherits properties and behaviors (methods) from a superclass.

Dependency

This is a relationship where one class relies on another in some way, often through method parameters, return types or temporary associations.

Summary

- **Association**: Objects are related but can exist independently.
- **Aggregation**: A weak "has-a" relationship where the contained objects can exist

independently.

- **Composition**: A strong "has-a" relationship where the contained objects cannot exist without the container.
- **Inheritance**: A subclass inherits from a superclass (is-a relationship).
- **Dependency**: One class depends on another for its functionality.
- **Realization**: A class implements the behavior defined by an interface.

How OOP Concepts Influence Design Patterns

- Encapsulation and Design Patterns:
  - Encapsulation forms the backbone of **Creational Patterns** like **Singleton** and **Builder**, where object creation details are hidden.
  - Example: Singleton encapsulates the creation and access control to a single instance of a class.
- Abstraction and Design Patterns:
  - Many patterns rely on abstraction, such as **Factory Pattern** and **Abstract Factory**, where the client interacts with abstract interfaces.
  - Example: Factory Pattern abstracts the creation of objects by delegating it to subclasses.
- Inheritance and Design Patterns:
  - Inheritance is key to **Structural Patterns** like **Adapter** and **Decorator**, where behavior is added or adapted using subclassing.
  - Example: **Decorator Pattern** uses inheritance to add responsibilities to objects at runtime.
- Polymorphism and Design Patterns:
  - **Behavioral Patterns** like **Strategy** and **Command** use polymorphism to define interchangeable behaviors.
  - Example: Strategy Pattern allows different algorithms (polymorphic behavior) to be swapped dynamically during runtime.

Module 2: SOLID Principles

Principle	Full Name	Purpose
S	Single Responsibility Principle (SRP)	One reason to change
O	Open/Closed Principle (OCP)	Open for extension, closed for modification
L	Liskov Substitution Principle (LSP)	Subtypes must be substitutable
I	Interface Segregation Principle (ISP)	Many client-specific interfaces
D	Dependency Inversion Principle (DIP)	Depend on abstractions, not concretions

---

1. Single Responsibility Principle (SRP)

What is SRP?

A class should have **only one reason to change** - meaning only one responsibility or job.

Bad Implementation (Violates SRP)

BadImpl/Invoice.java - Multiple responsibilities in one class:

```
public class Invoice {
    private double amount;

    public void generateInvoice() {
    }      // Responsibility 1: Generate

    public void saveToDatabase() {
    }       // Responsibility 2: Persist
}
```

Good Implementation (Follows SRP)

GoodImpl/Invoice.java - Only business logic:

```
public class Invoice {
    private double amount;

    public void generateInvoice() {
    }     // Single Responsibility
}
```

GoodImpl/InvoiceRepository.java - Database responsibility:

```
public class InvoiceRepository {
    public void saveToDatabase() {
    }      // Single Responsibility
}
```

GoodImpl/EmailService.java - Email responsibility:

```
public class EmailService {
    public void sendEmail() {
    }           // Single Responsibility
}
```

Golden Rule ??

If you can describe a class in one sentence without "and/or", it follows SRP.

---

2. Open/Closed Principle (OCP)

A class should be **open for extension** but **closed for modification**. Add new functionality through extension (inheritance/composition) without modifying existing code.

Bad Implementation (Violates OCP)

Problem: Must modify the existing method to add new payment types

PaymentProcessor.java — Hard-coded conditionals:

```
public class PaymentProcessor {
    /**
     * Issue: Adding new payment types [UPI...] requires modifying this method,
     * which violates OCP. This can lead to bugs and maintenance issues.
     */
    public void processPayment(String paymentType, double amount) {
        if (paymentType.equals("CreditCard")) {
            System.out.println("Processing credit card payment of amount: " + amount);
        } else if (paymentType.equals("DebitCard")) {
            System.out.println("Processing DebitCard payment of amount: " + amount);
        } else if (paymentType.equals("PayPal")) {
            System.out.println("Processing PayPal payment of amount: " + amount);
        } else {        throw new IllegalArgumentException("Unsupported payment type: " + paymentType);        }    }   }
```

✅ Good Implementation (Follows OCP)

Approach: Use interface (abstraction) → **Add new behavior via** new classes → **No modification to existing code**

Step 1: Create Interface

```
public interface Payment {
    void pay(double amount);
}
```

Step 2: Implement Payment Types

```
public class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Processing credit card payment: " + amount);
    }
}

public class DebitCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Processing debit card payment: " + amount);
    }
}

public class PayPalPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Processing PayPal payment: " + amount);
    }
}
```

Step 3: Use Abstraction in Processor

```
public class PaymentProcessor {

    public void processPayment(Payment payment, double amount) {
        payment.pay(amount);
    }
}

Main {
Payment payment = new CreditCardPayment();
PaymentProcessor processor = new PaymentProcessor();
processor.processPayment(payment, 1000);
}
```

Extend Without Modification

```
public class UpiPayment implements Payment {
    public void pay(double amount) {
```

```
        System.out.println("Processing UPI payment: " + amount);
    }
}
```

Key Idea

Add new classes → NO change in existing code Achieves true Open for Extension, Closed for Modification

---

3. Liskov Substitution Principle (LSP)

What is LSP?

Objects of a superclass should be replaceable with objects of its subclasses **without breaking the application**.

In simple terms:
☞ If class **B** is a subtype of class **A**, then we should be able to use **B** anywhere we use **A** without unexpected behavior.

Bad Implementation (Violates LSP)

BadImpl/Bird.java

```
public class Bird {
    public void fly() {
        System.out.println("Bird is flying");
    }
}
```

BadImpl/Ostrich.java

```
public class Ostrich extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Ostrich can't fly");
    }
}
```

Usage

```
public class Main {
    public static void main(String[] args) {
        Bird bird = new Ostrich();
        bird.fly(); // Runtime error ❌
    }
}
```

Problems:

- Subclass changes expected behavior
- Breaks polymorphism
- Causes runtime errors
- Violates trust in inheritance

Good Implementation (Follows LSP)

Class Hierarchy (Good Design)

```
Bird (abstract)
├── FlyingBird (abstract)
│   └── Sparrow
└── Ostrich
```

GoodImpl/Bird.java

```java
public abstract class Bird {
}
```

GoodImpl/FlyingBird.java

```java
public abstract class FlyingBird extends Bird {
    public void fly() {
        System.out.println("Flying...");
    }
}
```

GoodImpl/Sparrow.java

```java
public class Sparrow extends FlyingBird {
}
```

GoodImpl/Ostrich.java

```java
public class Ostrich extends Bird {
    // No fly method ✅
}
```

Usage

```java
public class Main {
    public static void main(String[] args) {
        FlyingBird bird = new Sparrow();
        bird.fly(); // Works fine ✅
    }
}
```

---

4. Interface Segregation Principle (ISP)

What is ISP?

Clients should not be forced to depend on methods they do not use. Prefer many specific interfaces over one general-purpose interface.

Bad Implementation (Violates ISP)

Problem: A single interface with multiple responsibilities forces all implementers to provide implementations for methods they don't need.

BadImpl/Worker.java

```
public interface Worker {
    void work();
    void eat();
}
```

BadImpl/HumanWorker.java

```
public class HumanWorker implements Worker {
    @Override
    public void work() {
        System.out.println("Human is working");
    }

    @Override
    public void eat() {
        System.out.println("Human is eating");
    }
}
```

BadImpl/RobotWorker.java

```
public class RobotWorker implements Worker {
    @Override
    public void work() {
        System.out.println("Robot is working");
    }

    @Override
    public void eat() {
        throw new UnsupportedOperationException("Robot can't eat");
    }
}
```

Problems:

- Robots are forced to implement eat(), leading to runtime errors or empty implementations
- Violates ISP by making interfaces too broad
- Hard to maintain and extend

Good Implementation (Follows ISP)

Approach

- Split the interface into smaller, more specific interfaces
- Clients implement only the interfaces they need

GoodImpl/Workable.java

```
public interface Workable {
    void work();
}
```

GoodImpl/Eatable.java

```
public interface Eatable {
    void eat();
}
```

GoodImpl/HumanWorker.java

```
public class HumanWorker implements Workable, Eatable {
    @Override
    public void work() {
        System.out.println("Human is working");
    }

    @Override
    public void eat() {
        System.out.println("Human is eating");
    }
}
```

GoodImpl/RobotWorker.java

```
public class RobotWorker implements Workable {
    @Override
    public void work() {
        System.out.println("Robot is working");
```

```
    }
}
```

Usage

```
public class Main {
    public static void main(String[] args) {
        Workable human = new HumanWorker();
        human.work(); // Human is working

        Workable robot = new RobotWorker();
        robot.work(); // Robot is working

        Eatable eater = new HumanWorker();
        eater.eat(); // Human is eating
    }
}
```

Key Idea

Separate interfaces → Clients depend only on what they use Achieves better decoupling and flexibility

---

5. Dependency Inversion Principle (DIP)

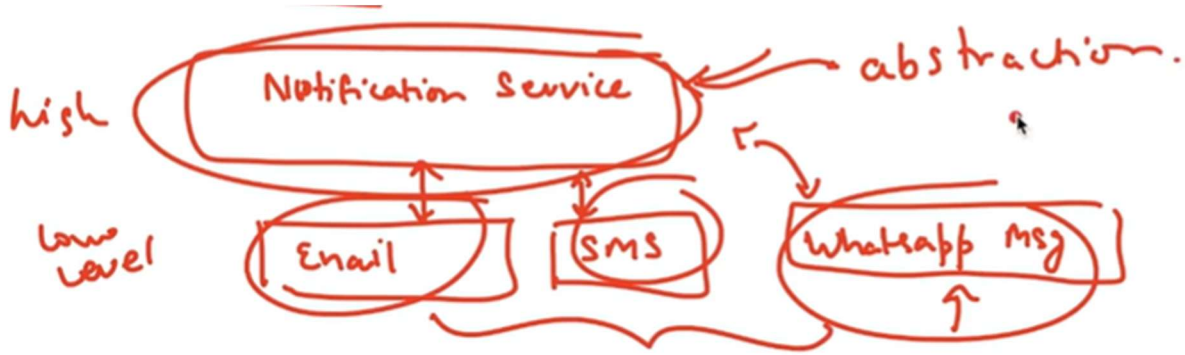
What is DIP?

High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces).

Abstractions should not depend on details. Details should depend on abstractions.

Bad Implementation (Violates DIP)

Problem: High-level module (NotificationManager) directly depends on low-level module (EmailService)



BadImpl/EmailService.java

```java
public class EmailService {
    public void sendEmail(String message) {
        System.out.println("Sending email: " + message);
    }
}
```

BadImpl/NotificationManager.java

```java
public class NotificationManager {
    private EmailService emailService;

    public NotificationManager() {
        this.emailService = new EmailService(); // Direct dependency
    }

    public void notifyUser(String message) {
        emailService.sendEmail(message);
    }
}
```

Problems:

- Tight coupling between NotificationManager and EmailService
- Hard to test NotificationManager without EmailService
- Difficult to change notification method (e.g., add SMS)

Good Implementation (Follows DIP)

Approach

- Introduce abstraction (interface)
- Both high-level and low-level modules depend on the abstraction

---

GoodImpl/NotificationService.java

```java
public interface NotificationService {
    void sendNotification(String message);
}
```

GoodImpl/EmailNotification.java

```java
public class EmailNotification implements NotificationService {
    @Override
    public void sendNotification(String message) {
        System.out.println("Sending email: " + message);
    }
}
```

GoodImpl/SMSNotification.java

```java
public class SMSNotification implements NotificationService {
    @Override
    public void sendNotification(String message) {
        System.out.println("Sending SMS: " + message);
    }
}
```

GoodImpl/NotificationManager.java

```java
public class NotificationManager {
    private NotificationService notificationService;

    public NotificationManager(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void notifyUser(String message) {
        notificationService.sendNotification(message);
    }
}
```

Usage

```java
public class Main {
    public static void main(String[] args) {
        NotificationService emailService = new EmailNotification();
        NotificationManager manager = new NotificationManager(emailService);
        manager.notifyUser("Hello via Email"); // Sending email: Hello via Email

        NotificationService smsService = new SMSNotification();
        NotificationManager smsManager = new NotificationManager(smsService);
        smsManager.notifyUser("Hello via SMS"); // Sending SMS: Hello via SMS
    }
}
```

Key Idea

Depend on abstractions → Loose coupling, easy to test and extend

DRY Principle : DRY(Don’t Repeat Yourself)

Avoid code duplication by reusing existing code and organizing it into reusable functions, classes, and modules. Why It Matters: Repeated code increases the risk of bugs, makes maintenance difficult, and increases the chance of inconsistencies. Example:Instead of writing the same database connection logic in multiple places, use a DatabaseConnection class that handles this logic, ensuring code reuse.

Module 3: Design Patterns

What are Design Patterns?

A design pattern is a general, reusable solution to a common problem within a specific context in software design.

Patterns are blueprints that can be applied in multiple situations, but they are not direct pieces of code.

They focus on object interactions, system architecture, and how classes/objects communicate.

Benefits of Design Patterns

Code Reusability: Patterns provide proven solutions that can be applied in multiple projects.

Maintainability: Encourages clean, understandable, and structured code, which is easier to maintain and extend.

Communication: Design patterns act as a common language between developers, improving communication and understanding.

Scalability: Design patterns help in structuring code to support future scaling with fewer refactors.

Efficiency: Avoids reinventing the wheel by using well-tested and widely accepted solutions.

Historical Context:

Origin (1970s-1980s): Early ideas of patterns began with Christopher Alexander in architecture.

His concepts of patterns in building design inspired the software industry.

Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice

Gang of Four (1994): The most famous book on design patterns,

"Design Patterns: Elements of Reusable Object-Oriented Software", was written by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (GoF). Introduced 23 classical design patterns.

Importance:

Promoted modular, maintainable, and scalable code. Encouraged reusability of solutions across different projects.

GoF Patterns: Many of the 23 patterns introduced in the Gang of Four book are still widely used today, e.g., Factory, Singleton, Observer.

Types of Design Patterns

- **Behavioural Patterns:** Focus on communication between objects and responsibilities. Examples: *Observer Pattern, Strategy Pattern, Command Pattern*
  - Behavioral Example: A notification system using the Observer Pattern, where various modules subscribe to system events.

- **Creational Patterns:** Deal with object creation mechanisms, trying to create objects in a manner suitable to the situation.
  - Examples: *Factory Pattern, Singleton Pattern, Builder Pattern*
  - Creational Example: A factory that produces different types of documents (Word, PDF, Excel) depending on the input parameters.

- **Structural Patterns:** Deal with the composition of objects or classes to form larger structures.
  - Examples: *Adapter Pattern, Composite Pattern, Decorator Pattern*
  - Structural Example: Using the Adapter Pattern to integrate a new API into an existing codebase.

Module 3.1: Behavioural Patterns

- Overview of Behavioral Design Patterns

1. Memento Pattern
2. Observer Pattern
3. Strategy Pattern
4. Command Pattern
5. Template Method Pattern
6. Iterator Pattern
7. State Pattern
8. Mediator Pattern

Overview of Behavioral Design Patterns

Behavioral patterns focus on how **objects communicate** and interact, managing the flow of information between entities**. (Object Interaction)**

They simplify complex control flow by defining clear communication and behavior among objects.

They provide solutions for managing **object relationships** and **communication protocols** to promote **loose coupling** and enhance flexibility.

Common Applications**:**

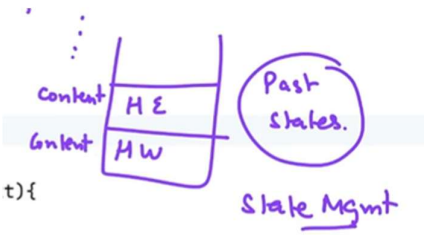
Coordinating **interactions** between objects.

Managing **state transitions** and **communication** efficiently.

1. Memento Pattern

Problem: How to provide **undo/redo** functionality or state restoration without exposing the object's internal state and breaking encapsulation.

Solution: The Memento Pattern captures the internal state of an object in a **memento**, allowing the object to restore its state later on without revealing internal details.



The **Memento Pattern** is a behavioral design pattern that captures and externalizes an object's internal state without violating encapsulation, allowing the object to be restored to this state later.

Why Use Memento Pattern?

- **Undo/Redo Functionality**: Save snapshots of object state for reverting to previous states
- **Encapsulation**: Capture state without exposing internal structure
- **History Management**: Maintain a history of changes without modifying the original object
- **Separation of Concerns**: Separate state management from the main object logic

---

Key Components

Component	Responsibility
Originator	Object whose state needs to be saved (TextEditor)
Memento	Captures and stores the internal state of the originator.(EditorMemento)
Caretaker	Manages and stores the mementos, without modifying them. (Caretaker)

Implementation

1. Memento Class (Store State)

EditorMemento.java - Immutable snapshot of state:

```java
// Memento clas : Stores the internal state of the TextEditor.
// Immutable snapshot
public class EditorMemento {

    private final String content;

    public EditorMemento(String content) {
        this.content = content;
    }

    public String getContent() {
        return content;
    }
}
```

Key Points:

- Immutable: State cannot be changed after creation
- Stores the snapshot of editor content
- Only accessible through getter method

---

2. Originator Class (Create State)

TextEditor.java - Creates and restores mementos:

Key Points:

- `save()`: Creates a memento with current state
- `restore()`: Restores from a memento
- Encapsulates the content management

```java
/**
 * A text editor that allows users to undo changes such as text addition,deletion or formating . The
 * text editor maintains a history of changes using the Memento design pattern, allowing users to
 * revert to previous states of the document.
 */
public class TextEditor {

  private StringBuilder content = new StringBuilder();

  // Append text
  public void append(String text) {
    content.append(text);
  }

  // Save current state
  public EditorMemento save() {
    String snapshot = content.toString();
    System.out.println(" Saving: " + snapshot);
    return new EditorMemento(snapshot);
  }

  // Restore previous state
  public void restore(EditorMemento memento) {
    if (memento == null) {
      System.out.println("Nothing to restore!");
      return;
    }
    content = new StringBuilder(memento.getContent());
    System.out.println("Restored to: " + content);
  }

  public String getContent() {
    return content.toString();
  }
}
```

---

3. Caretaker Class (Manage History)

Caretaker.java - Manages memento history:

```java
import java.util.Stack;

// Caretaker class: Manage mementos (snapshots of textEditor's state)
public class Caretaker {

  private final Stack<EditorMemento> history = new Stack<>();

  // Save state
  public void saveState(TextEditor editor) {
    history.push(editor.save());
  }
```

```java
  // Undo
  public void undo(TextEditor editor) {
    if (history.isEmpty()) {
      System.out.println(" Nothing to undo!");
      return;
    }

    history.pop(); // remove current
    editor.restore(history.peek()); // restore previous
  }
}
```

Key Points:

- Maintains a history stack of mementos
- `saveState()`: Saves current state to history
- `undo()`: Reverts to previous state
- No direct access to editor's internal state

Usage Example

TextEditorMain.java - Demonstrates undo functionality:

```java
public class TextEditorMain {

  public static void main(String[] args) {

    TextEditor editor = new TextEditor();
    Caretaker caretaker = new Caretaker();

    // Initial state
    editor.append("Hello");
    caretaker.saveState(editor);

    editor.append(" World");
    caretaker.saveState(editor);

    editor.append("!!!");
    caretaker.saveState(editor);

    System.out.println("\nCurrent: " + editor.getContent());

    // Undo operations
    System.out.println("\n--- Undo 1 ---");
    caretaker.undo(editor);
    System.out.println("Current: " + editor.getContent());

    System.out.println("\n--- Undo 2 ---");
    caretaker.undo(editor);
    System.out.println("Current: " + editor.getContent());   } }
```

Output

```
 Saving: Hello
 Saving: Hello World
 Saving: Hello World!!!

Current: Hello World!!!

--- Undo 1 ---
 Saving: Hello World
Restored to: Hello World
Current: Hello World

--- Undo 2 ---
 Saving: Hello
Restored to: Hello
Current: Hello
```

Real-World Analogies

- **Chess Game**: Save position after each move, restore previous position
- **Photo Editor**: Save layer states to enable undo operations
- **IDE Code Editor**: Track file history for undo/redo
- **Database**: Transaction rollback functionality
- Text editors (Undo/Redo operations)
- Database transaction rollbacks
- Game checkpoints and save states
- Document version control
- Snapshot-based state management

---

2. Observer Pattern

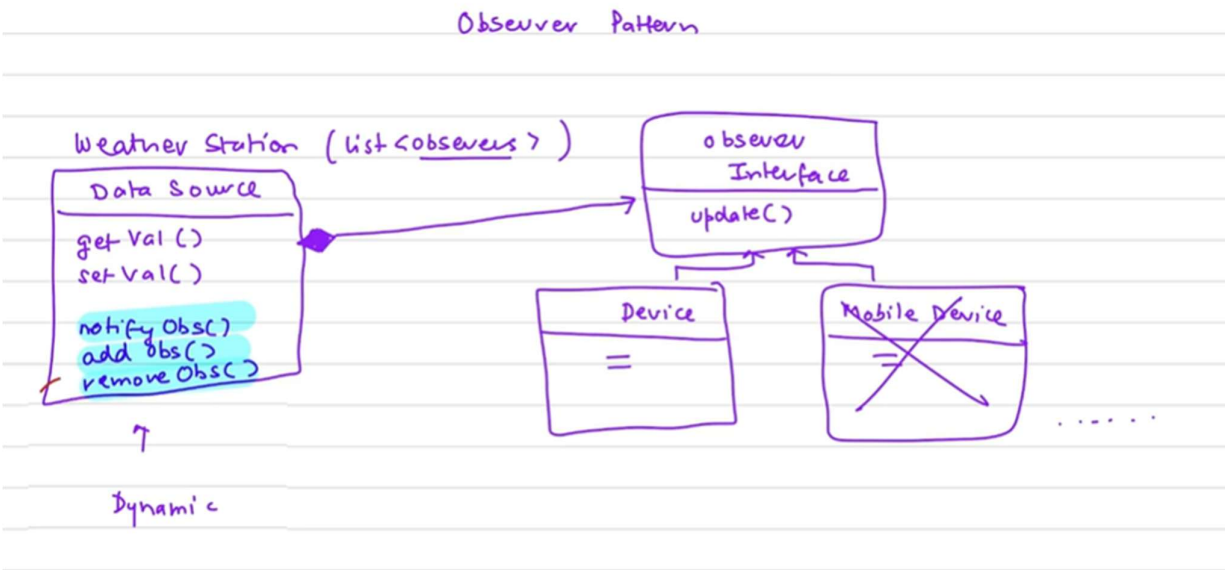
Suppose we have a weather station that records temperature and multiple devices (e.g., display units) want to show the latest temperature. Without using the observer pattern, the weather station would have to explicitly inform each device about the temperature change, which results in tight coupling between the station and devices.

The WeatherStationclass is tightly coupled to the DisplayDeviceclass.

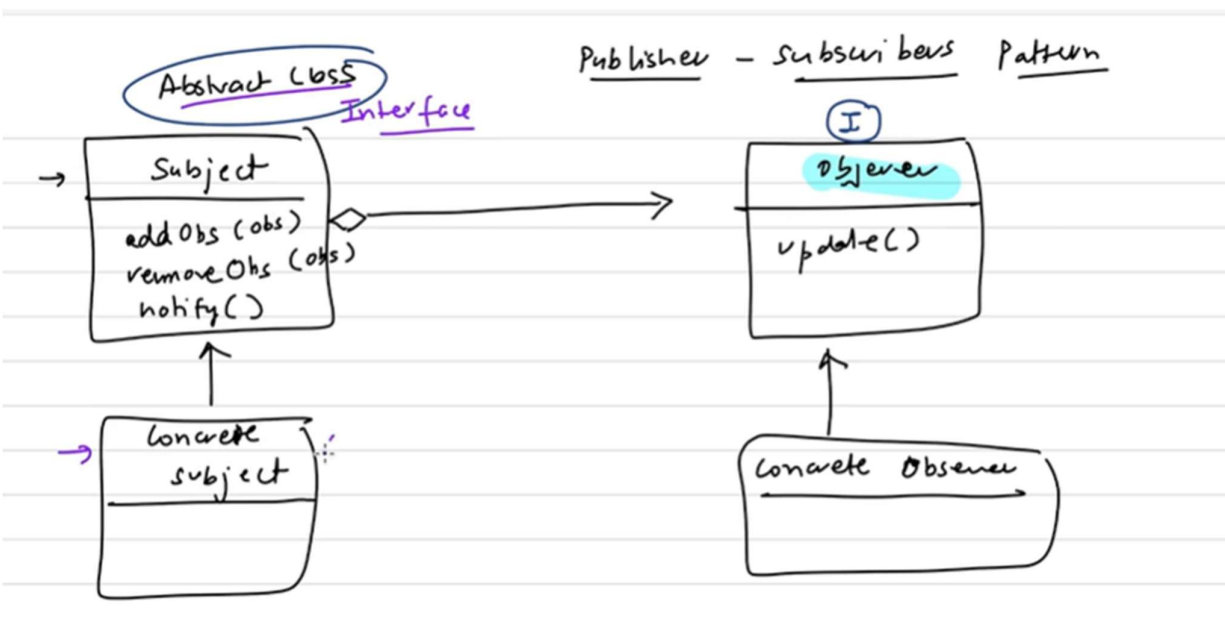
If we need multiple devices (e.g., mobile, tablet) to display the temperature, we need to modify the WeatherStationclass, leading to poor scalability and flexibility.

Observer Pattern

Problem: There is a need to notify multiple objects about a change in state without tightly coupling them.



Solution: The Observer Pattern defines a **one-to-many dependency**. When one object (the **Subject**) changes its state, all its dependents (the **Observers**) are notified and updated automatically. This pattern is widely used for implementing distributed event-handling systems, also known as the **publish-subscribe** pattern.



The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects so that when one object (subject) changes state, all its dependents (observers) are notified and updated automatically.

Why Use Observer Pattern?

- **Loose Coupling**: Subject and observers are loosely coupled - subject doesn't need to know concrete observer classes
- **Dynamic Relationships**: Observers can be added or removed at runtime
- **Broadcast Communication**: One subject can notify multiple observers simultaneously
- **Separation of Concerns**: Subject focuses on its core logic, observers handle their own updates

---

Key Components

Component	Responsibility
Subject	Interface/abstract class defining observer management methods
Observer	Interface defining update method for receiving notifications
Concrete Subject	Implements Subject, maintains state and notifies observers Maintains a list of observers and notifies them of changes.
Concrete Observer	Implements Observer, reacts to subject state changes Receive updates from the subject.

Implementation

1. Observer Interface

Observer.java - Defines the contract for observers:

```
// observer interface
interface Observer {
  void update(float temp);
}
```

Key Points:

- Defines `update()` method that concrete observers must implement
- Receives state change notifications from subject

- Parameter `temp` represents the changed state data

2. Subject Interface

Subject.java - Defines observer management:

```
// subject interface
interface Subject {
  void registerObserver(Observer o);

  void removeObserver(Observer o);

  void notifyObservers();
}
```

Key Points:

- `registerObserver()`: Adds observers to the notification list
- `removeObserver()`: Removes observers from the notification list
- `notifyObservers()`: Notifies all registered observers of state changes

3. Concrete Subject

WeatherStn.java - Weather station implementation:

```
// concrete subject = weather station
class WeatherStn implements Subject {

  private float temp;
  private List<Observer> observersList;

  public WeatherStn() {
    observersList = new ArrayList<>();
  }

  public void setTemp(float temp) {
    this.temp = temp;
    // notify all observers about the change in temperature
    notifyObservers();
  }

  @Override
  public void registerObserver(Observer o) {
    observersList.add(o);
  }

  @Override
  public void removeObserver(Observer o) {
    observersList.remove(o);
  }

  @Override
  public void notifyObservers() {
    for (Observer obs : observersList) {
```

```
    obs.update(temp); // Run-time polymorphism
  }         }             }
```

Key Points:

- Maintains temperature state (temp)
- Manages list of observers (observersList)
- setTemp() triggers notification when temperature changes
- notifyObservers() iterates through all observers and calls their update() method

DisplayDeviceIn.java - Display device observer:

```
class DisplayDeviceIn implements Observer {

  @Override
  public void update(float temp) {
    System.out.println("Temperature updated on displayDevice: " + temp);
  }
}
```

MobileApp.java - Mobile application observer:

```
class MobileApp implements Observer {

  @Override
  public void update(float temp) {
    System.out.println("Temperature updated on mobile: " + temp);
  }
}
```

Key Points:

- Both implement the Observer interface
- Each reacts differently to temperature updates
- Display device shows temperature on physical display
- Mobile app shows temperature in mobile interface

Usage Example

ObserverPatternSol.java - Demonstrates observer pattern:

```
public class ObserverPatternSol {
  public static void main(String[] args) {
    // Create a publisher
    WeatherStn station = new WeatherStn();

    // create subscribers
    DisplayDeviceIn display = new DisplayDeviceIn();
    MobileApp mobileApp = new MobileApp();

    // register subscribers to publisher
```

```
    station.registerObserver(display);
    station.registerObserver(mobileApp);

    // update temperature
    station.setTemp(30.5f);
    station.setTemp(22.5f);

    // Unregister a subscriber
    station.removeObserver(display);
    station.setTemp(27.0f);
  }
}
```

Output

```
Temperature updated on displayDevice: 30.5
Temperature updated on mobile: 30.5
Temperature updated on displayDevice: 22.5
Temperature updated on mobile: 22.5
Temperature updated on mobile: 27.0
```

Code Flow

1. Create WeatherStn and observers (DisplayDeviceIn, MobileApp)
2. Register observers with station
3. setTemp() updates temperature and calls notifyObservers()
4. notifyObservers() loops through observersList, calling update() on each
5. Observers display temperature updates
6. removeObserver() unsubscribes display device
7. Subsequent updates only notify remaining observers

Real-World Analogies

- **YouTube Channel**: Subscribers get notified when new video is uploaded
- **Email Lists**: Subscribers receive emails when newsletters are sent
- **Event Listeners**: GUI frameworks often use the Observer Pattern to implement event listeners for handling button clicks, input changes, etc.
- **Stock Price Monitoring**: When a stock price changes, multiple subscribers (like investors or systems) can be notified of the change.
- **News Publishing Systems**: News articles are published (subject), and subscribers (users) are notified whenever a new article is available.
- **Social Media Notifications**: Users can subscribe to updates from specific accounts, and when an account posts (subject), all followers (observers) are notified.
- **Logging Systems**: Different logging handlers can observe events and log them as needed, such as to the console, file, or remote server.

3. StrategyPattern

Let's consider a simple payment system where users can pay using different methods like credit cards or PayPal. Without the Strategy Pattern, you might use if-elseconditions to handle the different payment methods, leading to less maintainable and flexible code.

The PaymentServiceclass has multiple responsibilities (deciding the payment type and processing it).

Adding a new payment method requires modifying the PaymentService class.

The use of if-elseconditions can make the code harder to maintain as more payment types are added.

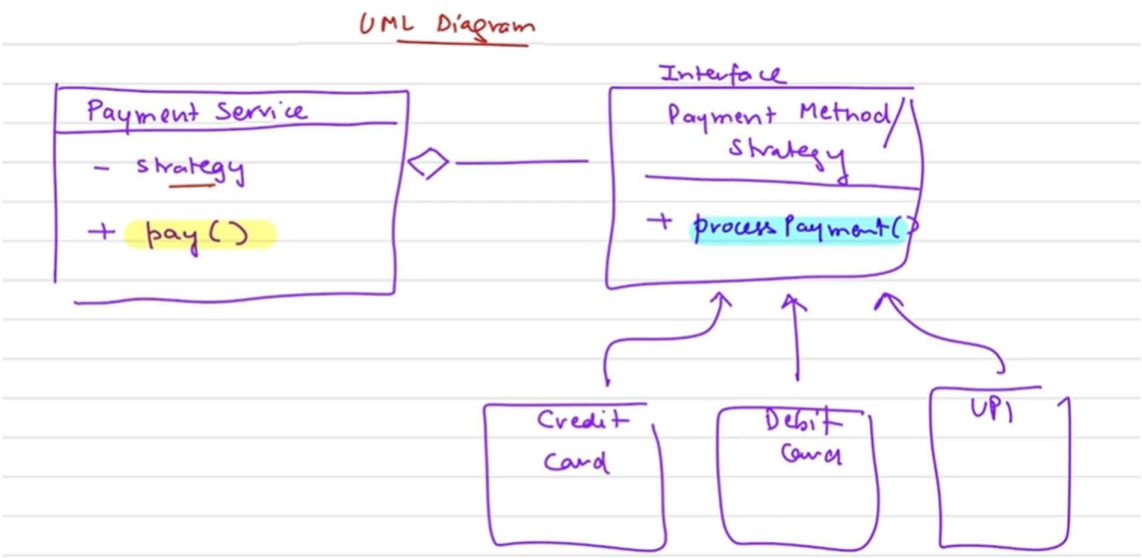
With the Strategy Pattern, the logic for each payment type is encapsulated in separate strategy classes, and the PaymentService(context class) delegates the task of payment processing to one of these strategies at runtime.

Strategy Pattern

- **Problem**: Hardcoded algorithms in classes lead to:
- Code **duplication**.
- Increased **maintenance complexity** when switching between algorithms.
- **Violation of Open/Closed Principle**: Modifications are required every time a new algorithm is introduced.
- **Solution**: The Strategy Pattern decouples the algorithm implementation from the client, allowing easy **switching of algorithms** without altering the client code.

Strategy Pattern Structure

- **Context**: The client class that uses a strategy to perform an operation.
- **Strategy Interface**: Defines the operations that all concrete strategies must implement.
- **Concrete Strategy**: Implements the actual algorithms, interchangeable based on the context.



Strategy Pattern Use Case

When to Use the Strategy Pattern:

When multiple algorithms need to be used interchangeably.

To avoid conditional statements (if-elseor switch-case) in the client code.

When a class has multiple behaviors, which can vary independently.

The PaymentServiceclass has multiple responsibilities (deciding the payment type and processing it).

Adding a new payment method requires modifying the PaymentService

class.

The use of if-elseconditions can make the code harder to maintain as more payment types are added.

With the Strategy Pattern, the logic for each payment type is encapsulated in separate strategy classes, and the PaymentService(context class) delegates the task of payment processing to one of these strategies at runtime.

Strategy Pattern

The **Strategy Pattern** is a behavioral design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from clients that use it.

Why Use Strategy Pattern?

- **Algorithm Encapsulation**: Different algorithms are encapsulated in separate classes
- **Runtime Flexibility**: Change algorithm behavior at runtime without modifying client code
- **Single Responsibility**: Each strategy class has one specific algorithm
- **Open/Closed Principle**: Add new strategies without changing existing code
- **Eliminate Conditional Logic**: Replace complex if-else chains with strategy objects

---

Key Components

Component	Responsibility
Strategy	Interface/common contract for all concrete strategies
Concrete Strategy	Implements specific algorithm/strategy
Context	Maintains reference to strategy and delegates work to it

---

Implementation

1. Strategy Interface

PaymentStrategy.java - Defines the strategy contract:

```
// interface for strategy
interface PaymentStrategy {
  void pay(int amount);
}
```

Key Points:

- Defines `pay()` method that all payment strategies must implement
- Takes `amount` parameter for payment processing
- Provides common interface for different payment methods

2. Concrete Strategies

CreditCardPayment.java - Credit card payment strategy:

```java
class CreditCardPayment implements PaymentStrategy {
  private String cardNumber;

  public CreditCardPayment(String cardNumber) {
    this.cardNumber = cardNumber;
  }

  @Override
  public void pay(int amount) {
    System.out.println("Paid " + amount + " using credit card: " + cardNumber);
  }
}
```

DebitCardPayment.java - Debit card payment strategy:

```java
class DebitCardPayment implements PaymentStrategy {
  private String cardNumber;

  public DebitCardPayment(String cardNumber) {
    this.cardNumber = cardNumber;
  }

  @Override
  public void pay(int amount) {
    System.out.println("Paid " + amount + " using debit card: " + cardNumber);
  }
}
```

Key Points:

- Both implement `PaymentStrategy` interface
- Each encapsulates specific payment logic
- Maintain their own state (card number)
- Provide different payment implementations

3. Context Class

PaymentService.java - Context that uses strategies:

```java
class PaymentService {
  private PaymentStrategy paymentStrategy;

  public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
    this.paymentStrategy = paymentStrategy;
  }

  public void processPayment(int amount) {
    if (paymentStrategy == null) {
      throw new IllegalStateException("Payment strategy not set");
    }
    paymentStrategy.pay(amount);
  }
}
```

Key Points:

- Maintains reference to current `PaymentStrategy`
- `setPaymentStrategy()` allows changing strategy at runtime
- `processPayment()` delegates to current strategy
- Includes validation to ensure strategy is set

Usage Example

StrategyPattern.java - Demonstrates strategy pattern:

```
public class StrategyPattern {
  public static void main(String[] args) {
    PaymentService paymentService = new PaymentService();

    // Using credit card payment strategy
    paymentService.setPaymentStrategy(new CreditCardPayment("1234-5678-9012-3456"));
    paymentService.processPayment(100);

    // Using debit card payment strategy
    paymentService.setPaymentStrategy(new DebitCardPayment("9876-5432-1098-7654"));
    paymentService.processPayment(200);
  }
}
```

Output

```
Paid 100 using credit card: 1234-5678-9012-3456
Paid 200 using debit card: 9876-5432-1098-7654
```

Code Flow

1. Create PaymentService context
2. Set CreditCardPayment strategy with card number
3. processPayment() delegates to strategy's pay() method
4. Change to DebitCardPayment strategy
5. processPayment() uses new strategy for payment

Real-World Analogies

- **Navigation Apps**: Choose route type (fastest, shortest, avoid tolls)
- **Text Editors**: Select sorting algorithm (alphabetical, date, size)

- **Media Players**: Choose compression format (MP3, AAC, FLAC)
- **E-commerce**: Select payment method (credit card, PayPal, bank transfer)
- **Travel Booking**: Choose transportation mode (flight, train, bus)

- Payment processing (credit card, PayPal, bank transfer)
- Sorting algorithms (quick sort, merge sort, bubble sort)
- Compression algorithms (ZIP, RAR, GZIP)
- Travel route planning (fastest, shortest, scenic routes)
- Text formatting (plain text, HTML, Markdown)

---

4. Command Pattern

Problem: Imagine you're developing a basic text editor with buttons for **bold**, **italic**, and

Underline text formatting.

Without the Command Pattern, the buttons directly interact with the TextEditor class, and you'd end up hardcoding behavior into the UI classes, making them tightly coupled.

Each button class is tightly coupled with the TextEditor. If the action changes, all button classes need to be modified.

It's harder to extend with new commands or add functionality such as undo/redo or logging.

Command Pattern

By introducing the Command Pattern, we can decouple the actions (bold, italic, underline) from the UI components (buttons), making the design more flexible and maintainable. The buttons no longer need to know about the editor directly but instead work with generic Command objects.

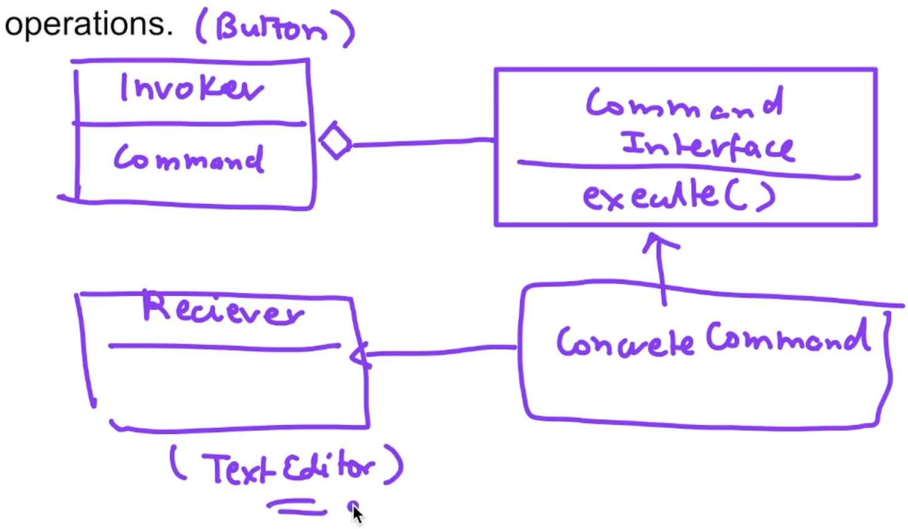
CommandPatternStructure

- **Command**: Interface for executing operations.
- **Invoker**: Sends the command.
- **Receiver**: Performs the operation.

CommandPatternBenefits

- **Decoupling of Invoker and Receiver**: The button (invoker) doesn't know the details of the TextEditor(receiver), making the system more flexible and reusable.
- **Command History and Undo**: Commands can be logged for undo/redo functionality.
- **Task Queuing**: Commands can be stored in a queue and executed later, making it useful for task scheduling.
- **Extensibility**: New commands can be added easily without modifying existing code. For example, adding a ChangeColorCommand only requires creating a new command class

UML Diagram



Command Pattern

The **Command Pattern** is a behavioral design pattern that encapsulates a request as an object, thereby allowing parameterization of clients with different requests, queuing of requests, and logging of the requests. It also enables undo/redo functionality.

Why Use Command Pattern?

- **Decoupling**: Decouples the object that invokes operation from the object that performs it
- **Encapsulation**: Packages requests as objects that can be passed around
- **Queuing**: Commands can be queued and executed at different times
- **Undo/Redo**: Facilitates implementation of undo/redo mechanisms
- **Logging**: Commands can be logged and replayed
- **Macro Recording**: Supports recording and playback of command sequences

Key Components

Component	Responsibility
Command	Interface defining the contract for command execution
Concrete Command	Implements specific operations and binds receiver
Receiver	Object that performs the actual work
Invoker	Object that triggers command execution

Implementation

1. Command Interface

Command.java - Defines command contract:

```
interface Command {
  void execute();
}
```

Key Points:

- Simple interface with `execute()` method
- All commands must implement this interface
- Provides uniform way to invoke different operations

---

2. Receiver Class

TextEditorCP.java - The object that performs actual work:

```
class TextEditorCP {
  public void bold() {
    System.out.println("Text is bolded");
  }

  public void italic() {
    System.out.println("Text is italicized");
  }

  public void underline() {
    System.out.println("Text is underlined");
  }
```

```
  public void changeColor() {
    System.out.println("Text color changed");
  }
}
```

Key Points:

- Contains the actual formatting operations
- Unaware of Command pattern
- Can be reused by different commands

---

3. Concrete Commands

BoldCommand.java - Encapsulates bold operation:

```
class BoldCommand implements Command {
  private TextEditorCP textEditor;

  public BoldCommand(TextEditorCP textEditor) {
    this.textEditor = textEditor;
  }

  @Override
  public void execute() {
    textEditor.bold();
  }
}
```

ChangeColorCommand.java - Encapsulates color change operation:

```
class ChangeColorCommand implements Command {
  private TextEditorCP textEditor;

  public ChangeColorCommand(TextEditorCP textEditor) {
    this.textEditor = textEditor;
  }

  @Override
  public void execute() {
    textEditor.changeColor();
  }
}
```

Key Points:

- Each command wraps a specific receiver operation
- Stores reference to receiver (TextEditorCP)
- Decouples invoker from receiver

4. Invoker Class

Button.java - Object that triggers commands:

```
class Button {
  private Command command;

  public void setCommand(Command command) {
    this.command = command;
  }

  public void click() {
    command.execute();
  }
}
```

Key Points:

- Maintains reference to a command
- `setCommand()` allows changing command at runtime
- `click()` invokes the command without knowing details

Usage Example

TextEditorWithCommandPattern.java - Demonstrates command pattern:

```
public class TextEditorWithCommandPattern {
  public static void main(String[] args) {
    TextEditorCP textEditor = new TextEditorCP();

    // One Button can have multiple commands
    Button boldButton = new Button();
    boldButton.setCommand(new BoldCommand(textEditor));
    boldButton.setCommand(new ChangeColorCommand(textEditor));

    // Simulate button click
    boldButton.click();
  }
}
```

Output

```
Text color changed
```

Note: Only the second command (ChangeColorCommand) is executed because setCommand() replaces the previous command.

Code Flow

1. Create TextEditorCP receiver with formatting operations
2. Create Button invoker
3. Wrap BoldCommand around bold() operation
4. Set BoldCommand on button
5. Replace it with ChangeColorCommand (color change operation)
6. Button click invokes current command's execute()

Real-World Analogies

- **Restaurant**: Customer (invoker) orders food (command), chef (receiver) prepares it
- **Smart Home**: App button (invoker) sends commands to smart devices (receivers)
- **Video Game**: Controller buttons (invoker) trigger game actions (commands)
- **Transaction Management**: Database commands queued and executed in sequence
- **Macro Recording**: Video editors record user actions as command sequences
- **GUI Applications:** Commands can be associated with buttons, menus, and keyboard shortcuts in applications like text editors, spreadsheets, or drawing software.
- **Task Scheduling:** Commands can be placed in a queue and executed later, useful in batch processing or deferred task execution.
- **Undo/Redo Functionality:** Commands can be stored and rolled back to provide undo and redo capabilities, especially in applications like IDEs, word processors, or graphics software.
- **Macro Recording:** Actions performed by the user can be recorded as a series of commands, which can then be played back as macros.

- **Text editor** operations (bold, italic, underline, undo, redo)
- **GUI button** clicks and menu items
- **Task scheduler** and job queuing systems
- Transaction management and rollback
- Remote invocation and network communication

CommandPatternDrawbacks

Increased Complexity: Introducing the Command Pattern can lead to more classes and complexity, especially when there are many different commands.

Overhead: Each operation becomes an object, which may add memory and performance overhead in systems with large numbers of commands

---

5. Template Method Pattern

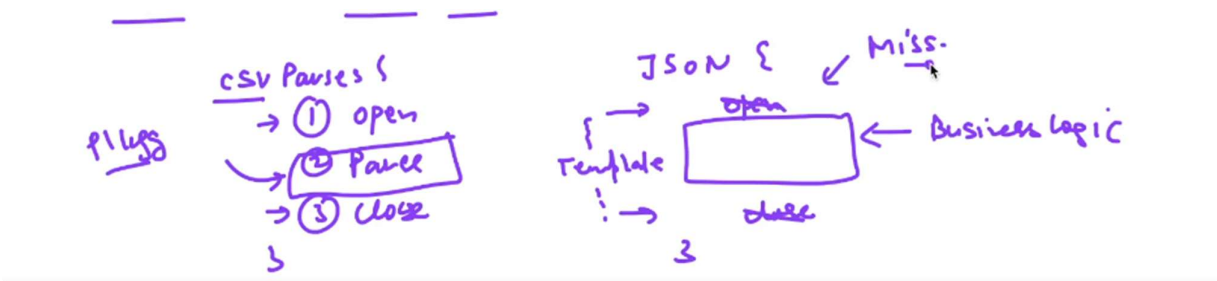
Consider a scenario where you have different data parsers (e.g., CSV, XML, and JSON). Each parser follows the same steps: **open file, parse data, and close file**.

Without the Template Method Pattern, you might end up duplicating the common steps in each parser class.

Problems in our code

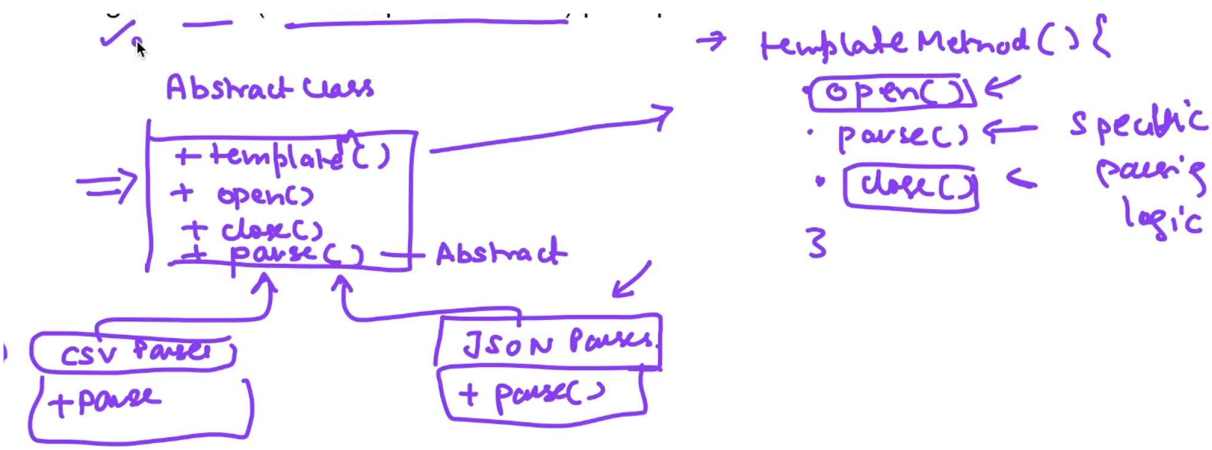
Code duplication: The openFile()and closeFile()methods are duplicated in both parsers.

Any changes to the common logic would require changes in every parser, violating the DRY (Don't Repeat Yourself) principle.



Problem: Different parts of an algorithm may need to vary in subclasses, but the overall structure should remain consistent.

Solution: The Template Method Pattern defines the **skeleton** of an algorithm in a base class and lets subclasses override specific steps.



Structure**:**

- **Abstract Class**: Defines the algorithm skeleton.
- **Concrete Subclasses**: Override specific steps of the algorithm.

Benefits

Code Reuse: Common code is moved to the parent class, promoting reuse and reducing duplication.

Flexibility: Subclasses can vary certain steps in the algorithm, while keeping the overall structure intact.

Consistency: Ensures that the high-level structure of the algorithm remains consistent, even when subclass behavior differs.

Use Cases of Template Method

UI Frameworks: Rendering a UI element might follow a fixed set of steps (initialize, draw, finish), but the details of how each element is drawn are left to subclasses.

Document Processing: A framework might define the skeleton for reading, processing, and saving documents, while specific formats (e.g., Word, PDF) provide their own processing logic.

Game Development: A game loop (initialize, update, render) can be defined in a base class, with specific games implementing their own logic for updating and rendering.

The **Template Method Pattern** is ideal for situations where a common algorithm exists, but some steps may need to be redefined by subclasses. It helps enforce structure and promotes reusability, while allowing flexibility where needed.

Template Pattern

The **Template Pattern** is a behavioral design pattern that defines the skeleton of an algorithm in a base class but lets subclasses override specific steps without changing the algorithm's structure. It follows the "Don't call us, we'll call you" principle.

Why Use Template Pattern?

- **Code Reuse**: Common algorithm steps are defined once in base class
- **Consistency**: Ensures consistent algorithm flow across implementations
- **Flexibility**: Allow subclasses to override only specific steps
- **Inversion of Control**: Subclasses implement specific behavior while base controls flow
- **Maintenance**: Changes to common steps affect all subclasses automatically
- **Variability**: Extract common behavior from similar algorithms

Key Components

Component	Responsibility
Abstract Class	Defines template method and abstract methods for subclasses
Template Method	Defines algorithm skeleton with fixed and variable steps
Primitive Operations	Abstract methods that subclasses must implement
Hook Methods	Optional methods subclasses can override (default implementations provided)
Concrete Classes	Implement specific variations of template steps

---

Implementation

1. Abstract Template Class

DataPraser.java - Defines algorithm skeleton:

```
abstract class DataPraser {

  public void parse(String data) {
    openFile();
    parseData();
    closeFile();
  }

  protected void openFile() {
    System.out.println("Opening file...");
  }

  protected void closeFile() {
    System.out.println("Closing file...");
  }

  protected abstract void parseData();
}
```

Key Points:

- `parse()` is the template method defining algorithm steps

- openFile() and closeFile() are concrete methods (hook methods)
- parseData() is abstract - must be implemented by subclasses
- Template method calls these methods in specific order

---

2. Concrete Implementations

CSVDataParser.java - CSV parsing implementation:

```
class CSVDataParser extends DataPraser {

  @Override
  protected void parseData() {
    System.out.println("Parsing CSV data...");
  }
}
```

JSONDataParser.java - JSON parsing implementation:

```
class JSONDataParser extends DataPraser {

  @Override
  protected void parseData() {
    System.out.println("Parsing JSON data...");
  }
}
```

Key Points:

- Both extend DataPraser abstract class
- Override only parseData() method
- Inherit common openFile() and closeFile() behavior
- Each provides specific parsing logic

Usage Example

WithTemplatePattern.java - Demonstrates template pattern:

```
public class WithTemplatePattern {
  public static void main(String[] args) {
    DataPraser csvParser = new CSVDataParser();
    csvParser.parse("data.csv");

    System.out.println();

    DataPraser jsonParser = new JSONDataParser();
    jsonParser.parse("data.json");
  }
}
```

Output

```
Opening file...
Parsing CSV data...
Closing file...

Opening file...
Parsing JSON data...
Closing file...
```

Code Flow

1. Create CSVDataParser instance
2. Call parse() template method with file path
3. Template executes: openFile() → parseData() → closeFile()
4. openFile() and closeFile() use inherited implementations
5. parseData() calls CSV-specific implementation
6. Repeat for JSONDataParser with JSON-specific implementation

---

Real-World Analogies

- **Recipe**: Template is the cooking process (read ingredients, mix, bake, serve), variations are different recipes
- **Manufacturing**: Assembly line (template) with station-specific tasks (variations)
- **Software Build Process**: Maven/Gradle define build lifecycle, plugins customize specific phases
- **Gaming**: Game loop (initialize, update, render) - same for all games, specific logic varies
- **Document Generation**: Same structure (header, body, footer), different content and formatting

- Data parsing (CSV, JSON, XML parsers)
- Document generation (PDF, HTML, Word formats)
- Database access patterns (JDBC templates)
- Game development (game loops)
- Build processes (maven, gradle)
- Cooking recipes (consistent steps, different ingredients)

---

6. Iterator Pattern

Iterator Pattern Motivation

Suppose you have a collection, such as an array or list, and you need to provide a mechanism for accessing its elements. Without the iterator pattern, the client code needs to understand how the collection is structured, and different collections would require different methods to traverse them.

Problems:

The client needs to know the internal structure of the collection (array in this case).

If we change the collection type (e.g., from an array to a linked list), we would need to modify the client code.

It's harder to implement different traversal strategies.

Iterator Pattern

Problem: How to access elements in a collection without exposing its internal representation.

Solution: The Iterator Pattern provides a way to **traverse** a collection without revealing its underlying structure, offering a uniform interface for traversal.

Structure**:**

Iterator: Interface for traversing a collection.

Collection: Holds the elements and provides an iterator.

Iterator Pattern Benefits

Separation of Concerns: The traversal logic is separated from the collection itself, allowing you to change one without affecting the other.

Uniform Interface: The same interface (Iterator) is used to traverse different types of collections, making the code more flexible.

Simplified Client Code: The client doesn't need to know the underlying data structure, reducing coupling and making the code easier to maintain.

Multiple Traversal Strategies: You can implement multiple types of iterators (e.g., forward, backward, filtered) without changing the collection.

Iterator Pattern Use Cases

Java Collections Framework: The Java Collections Framework (e.g., ArrayList, HashSet) uses the iterator pattern to provide a common interface (Iterator) for traversing different types of collections.

Database Cursors: In database programming, cursors are used to iterate over result sets. The iterator pattern can abstract this traversal, making it easier to work with data from a database without exposing the underlying query mechanism.

Tree Traversals: In tree data structures, the iterator pattern can be used to traverse nodes using different strategies like depth-first or breadth-first, without exposing the tree's internal structure.

File Systems: File systems can use the iterator pattern to traverse directories and files without exposing the internal details of how files and folders are stored.

Iterator Pattern Drawbacks

Additional Complexity: Implementing the iterator pattern can add extra layers of abstraction, especially for small or simple collections where direct traversal is sufficient.

Increased Overhead: For small collections or when the structure is unlikely to change, the overhead of creating iterators may not be justified.

Iterator Pattern

The **Iterator Pattern** is a behavioral design pattern that provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation. It allows traversal of collections without knowing their internal structure.

Why Use Iterator Pattern?

- **Encapsulation**: Access collection elements without exposing internal structure
- **Uniform Interface**: Same traversal interface for different collection types
- **Multiple Traversals**: Support multiple simultaneous traversals of same collection
- **Decoupling**: Separate collection storage from traversal logic
- **Extensibility**: Add new collection types without changing client code
- **Fail-Safe Iteration**: Prevent concurrent modification issues

Key Components

Component	Responsibility
Iterator	Interface defining traversal methods (hasNext, next, remove)
Concrete Iterator	Implements specific traversal logic for collection
Aggregate	Interface defining method to create iterator
Concrete Aggregate	Implements collection that creates appropriate iterator

Implementation

1. Element Class

Book.java - The element being stored in collection:

```java
public class Book {

  private String title;

  public Book(String title) {
    this.title = title;
  }

  public String getTitle() {
    return title;
  }

  public void setTitle(String title) {
    this.title = title;
  }

  @Override
  public String toString() {
    return "Book{" + "title='" + title + '\'' + '}';
  } }
```

Key Points:

- Simple data class representing collection elements
- Contains book title and getter/setter methods
- Unaware of Iterator pattern

---

2. Aggregate Interface

Iterable.java - Java's built-in interface for collections that can be iterated:

```java
public interface Iterable<T> {
    Iterator<T> iterator();
}
```

Key Points:

- Defines `iterator()` method that returns Iterator
- Generic interface allowing type-safe iteration
- Part of Java's standard library

---

3. Iterator Interface

Iterator.java - Java's built-in interface for iteration:

```java
public interface Iterator<E> {
    boolean hasNext();
    E next();
```

```
    default void remove() {
        throw new UnsupportedOperationException("remove");
    }
}
```

Key Points:

- hasNext(): Checks if more elements exist
- next(): Returns next element and advances position
- remove(): Optional removal of current element
- Generic interface for type safety

---

4. Concrete Aggregate

BookCollectionV3.java - Collection implementing Iterable:

```
public class BookCollectionV3 implements Iterable<Book>{

  private List<Book> books;

  public BookCollectionV3() {
    this.books = new ArrayList<>();
  }

  public void addBook(Book book) {
    books.add(book);
  }


  @Override
  public Iterator<Book> iterator() {
    return books.iterator();
  }
}
```

Key Points:

- Implements Iterable<Book> interface
- Uses ArrayList<Book> for internal storage
- iterator() returns ArrayList's built-in iterator
- Delegates iteration to underlying collection

Usage Example

ClientV3.java - Demonstrates iterator pattern:

```
public class ClientV3 {
  public static void main(String[] args) {
```

```
    BookCollectionV3 bookCollection = new BookCollectionV3();
    bookCollection.addBook(new Book("The Great Gatsby"));
    bookCollection.addBook(new Book("To Kill a Mockingbird"));
    bookCollection.addBook(new Book("1984"));

    Iterator<Book> iterator = bookCollection.iterator();
    while (iterator.hasNext()) {
      Book book = iterator.next();
      System.out.println(book.getTitle());
    }
  }
}
```

Output

```
The Great Gatsby
To Kill a Mockingbird
1984
```

Code Flow

1. Create BookCollectionV3 and add Book instances
2. Call iterator() to get Iterator instance
3. Use hasNext() to check for more elements
4. Call next() to retrieve current element and advance
5. Process element (print title)
6. Repeat until hasNext() returns false

Real-World Analogies

- **TV Remote**: Channel up/down buttons iterate through channels
- **Music Playlist**: Next/previous buttons traverse song list
- **File Explorer**: Navigate through directory structure
- **Database Cursor**: Move through query result rows
- **Social Media Feed**: Scroll through posts sequentially
- Collection traversal (List, Set, Map, custom collections)
- Database result sets
- File system directory traversal
- Tree/graph traversals
- Menu system navigation
- Playlist iteration in media players

7. State Pattern

You are tasked with building a **DirectionService**class for a navigation app. This class calculates the **estimated time of arrival (ETA)** and provides **directions** between two points. The ETA and direction differ based on the mode of transportation, which can be one of the following:

Walking, Cycling, Car, Train

Tight Coupling and Complex Conditional Logic:

The DirectionService likely uses **conditional statements** (if-elseor switch-case) based on transportation mode enums to determine how to calculate ETA and provide directions.

As the number of transportation modes increases, the conditional logic becomes **more complex and harder to maintain**.

Violation of the Open/Closed Principle:

Adding new transportation modes (e.g., Airplane, Boat) requires modifying the existing DirectionServiceclass, which goes against the **Open/Closed Principle** (classes should be open for extension but closed for modification).

Code Duplication and Reduced Maintainability:

Similar code blocks for different transportation modes may lead to **code duplication**, making the system less maintainable and more error-prone.

Scalability Issues:

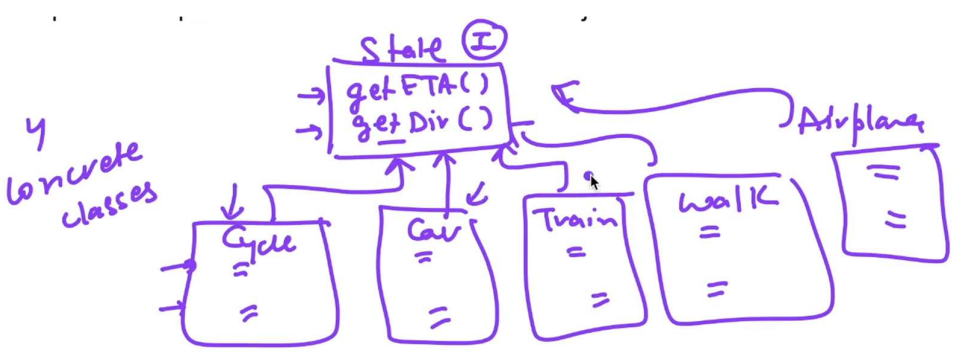
As more features or transportation modes are added, the class becomes **bulky**, impacting scalability and readability.

State Pattern: Structure

Context: Holds a reference to the current state.

State: Interface for state-specific behavior.

Concrete State: Specific implementations of the Stateinterface that represent a particular state of the context object.



State Pattern: Example

UI Navigation

Scenario: A mobile app UI where the navigation behavior changes based on whether the user is logged in or not.

Example**:**

States: **LoggedInState**, **LoggedOutState**.

Context: The app's navigation system switches between these states.

State Pattern

The **State Pattern** is a behavioral design pattern that allows an object to alter its behavior when its internal state changes. The object will appear to change its class. It encapsulates state-specific behavior and delegates to the current state object.

Why Use State Pattern?

- **State-Specific Behavior**: Different behavior for different states
- **Clean Code**: Eliminates complex conditional statements
- **Extensibility**: Add new states without modifying existing code
- **Single Responsibility**: Each state class handles one specific behavior
- **Runtime Flexibility**: Change object behavior at runtime
- **Maintainability**: State logic is localized and easy to modify

Key Components

Component	Responsibility
Context	Maintains reference to current state, delegates behavior to state
State	Interface/abstract class defining state-specific methods
Concrete State	Implements behavior for specific state

Implementation

1. State Interface

TransportationMode.java - Defines state contract:

```
public interface TransportationMode {
    int calcETA();
    String getDirection();
}
```

Key Points:

- Defines methods that all states must implement
- calcETA(): Calculate estimated time of arrival
- getDirection(): Get navigation directions
- Common interface for all transportation modes

2. Context Class

DirectionService.java - Context that maintains current state:

```
public class DirectionService {
  TransportationMode transportationMode;

  public DirectionService(TransportationMode transportationMode) {
    this.transportationMode = transportationMode;
  }

  public void setTransportationMode(TransportationMode transportationMode) {
    this.transportationMode = transportationMode;
  }

  // delegating the work current state's concrete class impl
  public int getETA() {
    return transportationMode.calcETA();
  }

  public String getDirection() {
    return transportationMode.getDirection();
  }
}
```

Key Points:

- Maintains reference to current TransportationMode state
- Constructor initializes with specific transportation mode
- setTransportationMode() allows runtime state changes
- getETA() and getDirection() delegate to current state
- Client interacts with context, unaware of specific states

---

3. Concrete States

Car.java - Car transportation state:

```
public class Car implements TransportationMode {
  @Override
  public int calcETA() {
    System.out.println("Calculating ETA for Car");
    return 15; // Example ETA in minutes
  }
```

```
  @Override
  public String getDirection() {
    System.out.println("Getting direction for Car");
    return "Head north on Main St, then turn right onto 1st Ave.";
  }
}
```

Train.java - Train transportation state:

```
public class Train implements TransportationMode {
  @Override
  public int calcETA() {
    System.out.println("Calculating ETA for Train");
    return 120; // Example ETA in minutes
  }

  @Override
  public String getDirection() {
    System.out.println("Getting direction for Train");
    return "Train direction: North";
  }
}
```

Cycling.java - Cycling transportation state:

```
public class Cycling implements TransportationMode {

  @Override
  public int calcETA() {
    System.out.println("Calculating ETA for cycling...");
    return 20; // Example ETA in minutes
  }

  @Override
  public String getDirection() {
    System.out.println("Getting directions for cycling...");
    return "Head north on Main St, then turn right onto 1st Ave.";
  }
}
```

Walking.java - Walking transportation state:

```
public class Walking implements TransportationMode {
  @Override
  public int calcETA() {
    System.out.println("Calculating ETA for walking...");
    return 30; // Example ETA in minutes
  }

  @Override
  public String getDirection() {
    System.out.println("Getting directions for walking...");
    return "Head north on Main St, then turn right onto 1st Ave.";
  }
}
```

Key Points:

- Each implements `TransportationMode` interface
- Provide state-specific ETA calculations and directions
- Car: Fastest (15 min), detailed street directions
- Train: Slowest (120 min), simple directional info
- Cycling: Medium speed (20 min), pedestrian-friendly routes
- Walking: Slow (30 min), detailed walking directions

Usage Example

WithStatePattern.java - Demonstrates state pattern:

```
public class WithStatePattern {
  public static void main(String[] args) {
    DirectionService service = new DirectionService(new Car());
    service.setTransportationMode(new Train());

    System.out.println(service.getETA());
    System.out.println(service.getDirection());
  }
}
```

Output

```
Calculating ETA for Train
120
Getting direction for Train
Train direction: North
```

Code Flow

1. Create DirectionService with initial Car state
2. Change state to Train using setTransportationMode()
3. Call getETA() - delegates to Train's calcETA() method
4. Call getDirection() - delegates to Train's getDirection() method
5. Each state provides different ETA and direction logic
6. Context remains unchanged, behavior changes with state

Real-World Analogies

- **Traffic Light**: Red, yellow, green states with different behaviors
- **Vending Machine**: Idle, has coin, dispensing, out of stock states

- **Document Workflow**: Draft, review, approved, published states
- **Game Character**: Idle, walking, running, jumping states
- **Order Processing**: Pending, processing, shipped, delivered states
- Document states (draft, review, published, archived)
- Game character states (idle, walking, running, jumping)
- Network connection states (connecting, connected, disconnected, error)
- Order processing states (pending, processing, shipped, delivered)
- UI component states (enabled, disabled, hover, pressed)
- Vending machine states (idle, has coin, dispensing, out of stock)
- **UI Components**: Buttons that change behavior based on state (enabled, disabled, pressed).
- **Vending Machines**: States like waiting for money, dispensing product, or out of stock.
- **TCP Connections**: Changing behavior based on connection state (listening, connected, closed).

---

8. Mediator Pattern

We want to build a chat system with multiple participants where each user can send messages to all other users. If users send messages to each other directly, the complexity increases as more users are added. Each user must know about every other user, creating a complex web of communication and dependencies.

Problems with Code

As more users are added, each user needs to manage direct communication with all others, leading to high coupling.

If a new communication rule is introduced (e.g., message logging), it would need to be added to all users.

Mediator Pattern

Problem: Objects in a system need to communicate, but direct communication leads to tight coupling and complexity.

Solution: The Mediator Pattern introduces a **mediator** object that handles all communication between objects, reducing direct dependencies and coupling.

In our chat app, by introducing a **Mediator** object, we will decouple the users from knowing about each other directly. The **Mediator** handles all communication, and the users (colleagues) only interact with the **Mediator**. This simplifies the interaction and reduces dependencies.

Mediator Pattern Code

ChatMediator Interface: Declares the sendMessage method, which all mediators must implement.

Concrete Mediator**:**

The ChatRoom class implements the ChatMediator interface. It holds a list of users and handles message broadcasting.

User Class**:**

Each User object represents a participant in the chat. When a user sends a message, the sendMessage method in the ChatRoom mediator is called, which distributes the message to all users except the sender.

Communication**:**

Users interact only with the ChatRoom (mediator), which facilitates communication between them, removing direct dependencies between individual users.

Mediator Pattern Benefits

Reduces Complexity: The mediator centralizes communication, reducing direct dependencies between objects.

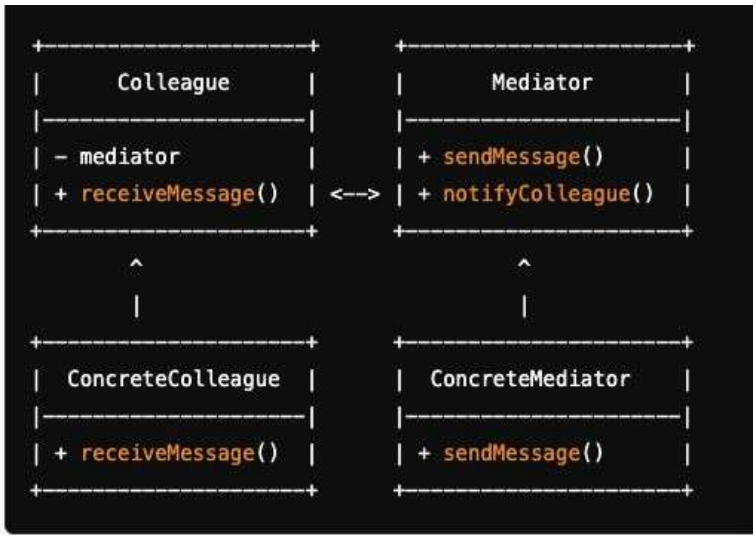
Loose Coupling: Colleagues only interact with the mediator, making them easier to manage, extend, and maintain.

Single Responsibility: The mediator handles complex communication logic, allowing colleagues to focus on their own behavior.

Centralized Control: Changes to communication rules can be made in the mediator without affecting the colleagues.

Mediator Pattern in Real World

UML Diagram



Mediator Pattern

The **Mediator Pattern** is a behavioral design pattern that defines how a set of objects interact with each other. Instead of objects communicating directly, they communicate through a mediator object. This promotes loose coupling by keeping objects from referring to each other explicitly.

Why Use Mediator Pattern?

- **Loose Coupling**: Objects don't need to know about each other
- **Centralized Control**: Communication logic is centralized in mediator
- **Maintainability**: Changes to interaction logic affect only the mediator
- **Reusability**: Colleague objects can be reused with different mediators
- **Simplified Dependencies**: Reduces complex many-to-many relationships
- **Encapsulation**: Interaction rules are encapsulated in one place

---

Key Components

Component	Responsibility
Mediator	Interface defining communication contract
Concrete Mediator	Implements communication logic, coordinates colleagues
Colleague	Objects that communicate through mediator

Implementation

1. Mediator Interface

ChatMediator.java - Defines mediator contract:

```
interface ChatMediator {
  void sendMessage(String message, ChatUser user);

  void addUser(ChatUser user);
}
```

Key Points:

- `sendMessage()`: Broadcast message to other users
- `addUser()`: Register new users with mediator
- Defines communication protocol for chat system

2. Concrete Mediator

ChatRoom.java - Implements chat coordination:

```
class ChatRoom implements ChatMediator {
  private List<ChatUser> users;

  public ChatRoom() {
    this.users = new ArrayList<>();
  }

  @Override
  public void sendMessage(String message, ChatUser sender) {
    for (ChatUser u : users) {
      if (u != sender) {
        u.receiveMessage(message, sender);
      }
    }
  }

  @Override
  public void addUser(ChatUser user) {
    users.add(user);
  }
}
```

Key Points:

- Maintains list of all chat users (colleagues)
- `sendMessage()` broadcasts to all users except sender
- `addUser()` registers new participants
- Centralizes all communication logic

---

3. Colleague Class

ChatUser.java - Chat participants:

```
class ChatUser {
  private String name;
  private ChatMediator chatMediator;

  public ChatUser(String name, ChatMediator chatMediator) {
    this.name = name;
    this.chatMediator = chatMediator;
  }

  public String getName() {
    return name;
  }

  public void sendMessage(String message) {
```

```
    System.out.println(this.name + " sends message: '" + message + "'");
    chatMediator.sendMessage(message, this);
  }

  public void receiveMessage(String message, ChatUser sender) {
    System.out.println(this.name + " receives message: '" + message + "' from " + sender.getName());
  }
}
```

Key Points:

- Each user has reference to mediator (not other users)
- `sendMessage()` delegates to mediator instead of direct communication
- `receiveMessage()` called by mediator when messages arrive
- Users communicate only through mediator

Usage Example

WithMediatorPattern.java - Demonstrates mediator pattern:

```
public class WithMediatorPattern {
  public static void main(String[] args) {

    ChatMediator chatRoom = new ChatRoom();

    ChatUser alice = new ChatUser("Alice", chatRoom);
    ChatUser bob = new ChatUser("Bob", chatRoom);
    ChatUser charlie = new ChatUser("Charlie", chatRoom);

    chatRoom.addUser(alice);
    chatRoom.addUser(bob);
    chatRoom.addUser(charlie);

    alice.sendMessage("Hello everyone!");
  }
}
```

Output

```
Alice sends message: 'Hello everyone!'
Bob receives message: 'Hello everyone!' from Alice
Charlie receives message: 'Hello everyone!' from Alice
```

Code Flow

1. Create ChatRoom mediator instance
2. Create ChatUser instances with reference to mediator
3. Register all users with mediator using addUser()
4. Alice calls sendMessage() which delegates to mediator
5. Mediator broadcasts message to all users except sender
6. Bob and Charlie receive the message through mediator

Real-World Analogies

- **Air Traffic Control**: Planes communicate through control tower
- **Chat Room**: Users communicate through chat server
- **UN Security Council**: Countries communicate through mediator
- **Event Bus**: Components communicate through central event system
- **Smart Home Hub**: Devices communicate through central controller
- Chat applications and messaging systems
- GUI dialog boxes with multiple controls
- Air traffic control systems
- Workflow management systems
- Event handling in complex UIs
- Multiplayer game coordination
- Smart home automation systems
- Air Traffic Control**:** Airplanes communicate through a central control tower (mediator) instead of coordinating directly with each other.
- GUI Component Coordination**:** In GUI applications, multiple UI components may need to interact. For example, when a dropdown changes, it may trigger updates to text fields, buttons, etc. A mediator can handle this interaction logic instead of having the components know about each other directly.
- Workflow Systems**:** In a business process management system, a mediator can coordinate various activities across multiple systems or departments.

Summary : This module covers key **Behavioral Design Patterns** that help define how objects interact and manage state transitions within a system. From **observer-based event notification** to **decoupling algorithms with strategies**, each pattern offers a solution to common communication challenges in software design.

Module 3.2: Creational Patterns

- Overview of Creational Design Patterns

1. Singleton Pattern
2. Builder Pattern
3. Prototype Pattern
4. Factory Pattern
5. Abstract Factory Pattern

Overview of Creational Design Patterns

Purpose: Creational patterns are focused on **object creation mechanisms**, aiming to optimize the creation process while ensuring flexibility.

Goal: They abstract the instantiation process to make systems **more flexible and reusable**.

Problem They Solve: Prevent tight coupling between code and object creation logic, simplifying the management of new object creation, especially in complex systems.

Example**:**

In a system where different types of documents (PDF, Word, Excel) are created, instead of using neweverywhere, a **Factory Pattern** can centralize object creation.

Application: They are applied in scenarios where:

You want to **separate the instantiation** process from usage.

The exact type of object needed can vary depending on runtime conditions.

1. Singleton Design Pattern

Motivation

In certain situations, such as managing a database connection, logging, or configuration settings, you want to ensure that only one instance of a class is created throughout the application's lifecycle. If multiple instances were created, it could lead to issues like:

Inconsistent state: If multiple instances represent the same concept, they may hold different data.

Resource conflicts: If multiple instances of a resource-heavy class are created, it can lead to performance degradation.

Singleton Design Pattern

The Singleton pattern is used when exactly **one instance** of a class is required to coordinate actions across the system.

When to Use

Global resource management (e.g., managing database connections, logging).

Configuration settings in applications that need to be shared.

Singleton Design Code

Private Constructor: The constructor is private so that no other class can instantiate AppSettings directly.

Singleton Access: The getInstance()method is used to access the single

instance of the settings.

Global Access: Any part of the application can access the settings using

AppSettings.getInstance().

Thread Safety in Singleton DP

Multi-threading issues: Without thread safety, multiple instances of the Singleton class could be created in a multi-threaded environment.

Solutions**:**

Lazy Initialization with Synchronization: Synchronize the method that creates the instance.

Double-Checked Locking: Optimize the performance by only locking when necessary.

Bill Pugh Singleton Design: Uses an inner static helper class to ensure thread safety and lazy loading.

Singleton Pattern

The **Singleton Pattern** is a creational design pattern that ensures a class has **only one instance** and provides a global point of access to it. It restricts object instantiation to a single object, preventing multiple instances from being created.

Why Use Singleton Pattern?

- **Single Instance**: Ensures only one object exists throughout application lifetime
- **Global Access Point**: Provides global point of access without parameter passing
- **Lazy Initialization**: Create instance only when first needed
- **Resource Control**: Manage expensive resources (database connections, thread pools)
- **Consistency**: Guarantee state consistency across application
- **Thread Safety**: Prevent race conditions in multi-threaded environments

Key Components

Component	Responsibility
Private Constructor	Prevent external instantiation
Static Instance	Hold the single instance of the class
Static Access Method	Provide global access to instance

Implementation

1. Singleton Class with Thread Safety

Singleton.java - Thread-safe singleton pattern:

```
class Singleton {
  // 1. Create static obj
  static Singleton singletonObject;

  // 2. Make private Constructor - using this we can't create outside its own class using new
  private Singleton() {
    System.out.println("Instance Created");
  }

  // 3. Make method to access singleton object created
  public static synchronized Singleton getInstance() {
    if (singletonObject == null) {
      System.out.println("1st thread is creating instance");
      singletonObject = new Singleton();
    } else {
      System.out.println("Instance already created by 1st thread, returning existing instance");
    }
    return singletonObject;
  }
}
```

Key Points:

- `static Singleton singletonObject`: Holds the single instance
- `private Singleton()`: Prevents instantiation from outside
- `synchronized getInstance()`: Ensures thread-safe lazy initialization
- `if (singletonObject == null)`: Creates instance only once
- Subsequent calls return existing instance

Thread-Safe Variants

Approach 1: Synchronized Method (Shown Above)

- Pros: Simple, lazy initialization
- Cons: Performance overhead on every call

Approach 2: Double-Checked Locking

```
public static Singleton getInstance() {
  if (singletonObject == null) {
    synchronized (Singleton.class) {
      if (singletonObject == null) {
        singletonObject = new Singleton();
      }
    }
  }
  return singletonObject;
}
```

Approach 3: Eager Initialization

```
static Singleton singletonObject = new Singleton();

public static Singleton getInstance() {
  return singletonObject;
}
```

Approach 4: Bill Pugh Singleton (Best Practice)

```
class Singleton {
  private Singleton() {}

  static class SingletonHolder {
    static Singleton instance = new Singleton();
  }

  public static Singleton getInstance() {
    return SingletonHolder.instance;
  }
}
```

Usage Example

Main.java - Demonstrates thread-safe singleton:

```
public class Main {
  public static void main(String[] args) {

    Thread t1 =
        new Thread(
            new Runnable() {
              @Override
              public void run() {
                Singleton obj = Singleton.getInstance();
              }
            });

    Thread t2 =
        new Thread(
            new Runnable() {
              @Override
              public void run() {
                Singleton obj = Singleton.getInstance();
              }
            });
    t1.start();
    t2.start();
  }
}
```

Output

```
1st thread is creating instance
Instance Created
Instance already created by 1st thread, returning existing instance
```

Code Flow

1. Two threads call getInstance() simultaneously
2. Synchronized lock prevents both from entering critical section
3. First thread creates instance and exits
4. Second thread checks condition, finds instance exists
5. Returns existing instance without creating new one
6. Both threads work with same singleton instance

Real-World Analogies

- **Government**: Only one president of a country at a time
- **Database Connection Pool**: Single pool manages all connections
- **Logger**: Single logger instance logs all application events
- **Cache Manager**: Single cache instance serves entire application
- **Configuration Manager**: Single config holds application setting
- **Theme Manager**: Single theme manager controls app appearance
- Thread pool managers
- Application settings holder
- Session managers
- Text formatting utilities

---

2. Builder Design Pattern

When an object requires many parameters, especially optional ones, the constructor can become hard to use or maintain. This issue can lead to:

Long constructor parameter lists.

Difficulty in understanding which values are optional or required.

Lack of flexibility when it comes to setting only some values.

For example, constructing an object with multiple optional parameters without the Builder pattern can look like this:

Problem: When a class constructor has too many parameters, the **Builder Pattern**

allows step-by-step construction of complex objects.

Solution: Separates the construction of an object from its representation, offering a fluent interface for creating complex objects

Builder Design Pattern

The **Builder Pattern** is a creational design pattern that separates the construction of a complex object from its representation, allowing step-by-step object creation. It enables flexible and readable construction of objects with many optional parameters, making code more maintainable and less error-prone.

- **Readable Code**: Fluent API makes object construction clear and self-documenting
- **Flexible Construction**: Build objects step-by-step with optional parameters
- **Avoid Telescoping Constructors**: Eliminate constructor overloading hell
- **Immutability**: Create immutable objects safely without huge constructor parameters
- **Object Consistency**: Ensure all required fields are set before object creation

- **Separation of Concerns**: Separates construction logic from business logic
- **Easier Maintenance**: Changes to object structure don't affect client code

Key Components

Component	Responsibility
Product	Complex object being constructed
Builder	Abstract builder interface with build methods
ConcreteBuilder	Implements builder methods to construct product
Director (Optional)	Controls construction sequence

Implementation

1. Basic Builder Pattern

House.java - Product class with inner builder:

```
class House {
  // Required parameters
  private String foundation;
  private String walls;
  private String roof;

  // Optional parameters
  private boolean hasGarage;
  private boolean hasSwimmingPool;
  private boolean hasGarden;

  // Private constructor - only accessible via builder
  private House(HouseBuilder builder) {
    this.foundation = builder.foundation;
    this.walls = builder.walls;
    this.roof = builder.roof;
    this.hasGarage = builder.hasGarage;
    this.hasSwimmingPool = builder.hasSwimmingPool;
    this.hasGarden = builder.hasGarden;
  }

  @Override
  public String toString() {
    return "House{" +
      "foundation='" + foundation + '\'' +
      ", walls='" + walls + '\'' +
      ", roof='" + roof + '\'' +
      ", hasGarage=" + hasGarage +
      ", hasSwimmingPool=" + hasSwimmingPool +
```

```
    ", hasGarden=" + hasGarden +
    '}';
  }

  // Inner Builder class
  static class HouseBuilder {
    // Required
    private String foundation;
    private String walls;
    private String roof;

    // Optional
    private boolean hasGarage = false;
    private boolean hasSwimmingPool = false;
    private boolean hasGarden = false;

    // Constructor with required parameters
    public HouseBuilder(String foundation, String walls, String roof) {
      this.foundation = foundation;
      this.walls = walls;
      this.roof = roof;
    }

    // Fluent builder methods for optional parameters
    public HouseBuilder setHasGarage(boolean hasGarage) {
      this.hasGarage = hasGarage;
      return this;
    }

    public HouseBuilder setHasSwimmingPool(boolean hasSwimmingPool) {
      this.hasSwimmingPool = hasSwimmingPool;
      return this;
    }

    public HouseBuilder setHasGarden(boolean hasGarden) {
      this.hasGarden = hasGarden;
      return this;
    }

    // Build method returns the constructed House
    public House build() {
      return new House(this);
    }
  }
}
```

Key Points:

- Product constructor is `private` - only builder can instantiate
- Builder provides fluent interface (methods return `this`)
- Required parameters in builder constructor
- Optional parameters have setter methods
- `build()` method creates final product

Builder Variants

Approach 1: Inner Builder (Shown Above)

- Pros: Simple, encapsulated, fluent API
- Cons: Tight coupling between product and builder
- Best for: Single product types

Approach 2: Separate Builder Class

```
interface Builder {
  Builder setFoundation(String foundation);
  Builder setWalls(String walls);
  Builder setRoof(String roof);
  House build();
}

class HouseBuilder implements Builder {
  private String foundation, walls, roof;
  private boolean hasGarage, hasSwimmingPool;

  @Override
  public Builder setFoundation(String foundation) {
    this.foundation = foundation;
    return this;
  }

  @Override
  public House build() {
    return new House(foundation, walls, roof, hasGarage, hasSwimmingPool);
  }
}
```

Approach 3: Director Pattern

```
class HouseDirector {
  private Builder builder;

  public HouseDirector(Builder builder) {
    this.builder = builder;
  }

  public House buildStandardHouse() {
    return builder
      .setFoundation("Concrete")
      .setWalls("Brick")
      .setRoof("Tile")
      .build();
  }
```

```
  public House buildLuxuryHouse() {
    return builder
      .setFoundation("Marble")
      .setWalls("Stone")
      .setRoof("Slate")
      .setHasGarage(true)
      .setHasSwimmingPool(true)
      .build();
  }
}
```

Usage Example

Client.java - Demonstrates builder pattern usage:

```
public class Client {
  public static void main(String[] args) {

    // Simple house
    House BasicHouse = new House.HouseBuilder("Concrete", "Brick", "Tile")
      .build();
    System.out.println("Basic House: " + BasicHouse);

    // House with garage
    House HouseWithGarage = new House.HouseBuilder("Concrete", "Brick", "Tile")
      .setHasGarage(true)
      .build();
    System.out.println("House with Garage: " + HouseWithGarage);

    // Luxury house
    House LuxuryHouse = new House.HouseBuilder("Marble", "Stone", "Slate")
      .setHasGarage(true)
      .setHasSwimmingPool(true)
      .setHasGarden(true)
      .build();
    System.out.println("Luxury House: " + LuxuryHouse);
  }
}
```

Output

```
Basic House: House{foundation='Concrete', walls='Brick', roof='Tile', hasGarage=false, hasSwimmingPool=false, hasGarden=false}
House with Garage: House{foundation='Concrete', walls='Brick', roof='Tile', hasGarage=true, hasSwimmingPool=false, hasGarden=false}
Luxury House: House{foundation='Marble', walls='Stone', roof='Slate', hasGarage=true, hasSwimmingPool=true, hasGarden=true}
```

Code Flow

1. Client creates builder with required parameters

2. Client calls optional setter methods in any order
3. Each setter method returns builder instance (fluent interface)
4. Client calls `build()` method to create product
5. Builder validates and constructs final product
6. Client receives complete, consistent object

Comparison: Constructor vs Builder

Aspect	Traditional Constructor	Builder Pattern
Readability	Poor with many params	Excellent, self-documenting
Optional Params	Telescoping constructors	Fluent setter methods
Flexibility	Low	High
Immutability	Easy but verbose	Better balance
Code Lines	Few	Many
Maintainability	Moderate	Good
Validation	In constructor	In builder

Real-World Analogies

- **House Construction**: Foundation→Walls→Roof→Finishing (step-by-step)
- **Car Manufacturing**: Frame→Engine→Interior→Paint (assembly line)
- **Meal Preparation**: Base→Protein→Vegetables→Sauce→Seasoning
- **Computer Assembly**: Case→Motherboard→CPU→Memory→Storage
- **Document Creation**: Title→Content→Formatting→Metadata
- **API Request Building**: Method→URL→Headers→Body→Query Params
- Building configuration objects with many optional parameters
- Creating complex domain objects (House, Computer, Resume)
- UI components with multiple customization options
- Database query builders
- Request/Response builders in REST APIs
- HTML/XML document builders
- Report builders with various options
- Game object creation with many attributes

3. Prototype Pattern

Problem

Consider a board game where you need to save the current state of the game at various checkpoints. Instead of manually creating new board objects and copying all the pieces or their states (which could be costly if the board is large and has many game pieces), we can use the Prototype Pattern to clone the board.

We will use the prototype pattern allows us to make a copy of the current board, including all its pieces and their states, without the need for deeply recreating each part of the board.

The **Prototype Pattern** can be extremely useful in a board game when you want to save the current state of the game (including the board layout and the position of pieces) for undo/redo functionality, checkpoints, or simply making a copy of the board for a new player.

Each piece or game element can provide its own clonemethod, allowing the entire board to be easily cloned with all its current state.

By using the Prototype pattern, we decouple the complexity of cloning the board from the client, ensuring that each object knows how to clone itself, making the system flexible and easier to maintain.

Shallow vs Deep Copy

Shallow Copy: Creates a new object but **does not clone the objects** that the original object refers to.

Deep Copy: Clones the original object and all the objects it refers to (nested objects)

Example:

Shallow Copy: Cloning an object with references to other objects (only the outer object is copied).

Deep Copy: Cloning the entire object graph, including any objects the original refers to.

Benefits

Simplifies Object Creation: Instead of manually copying each object, the clonemethod in each object simplifies the creation of copies.

Avoids Subclassing: The pattern relies on delegation to the clonemethod, allowing the class itself to handle object creation, avoiding the need for subclassing.

Shallow or Deep Copy: Depending on the use case, you can either implement a shallow copy (copying references) or a deep copy (cloning objects) based on the specific requirements.

Efficient Creation: When creating objects with a complex structure or when performance is a concern, the Prototype pattern allows you to efficiently replicate objects.

Consistency: Ensures that all properties of the object are consistently copied, avoiding errors associated with manual copying.

Prototype Design Pattern

The **Prototype Pattern** is a creational design pattern that creates new objects by copying an existing object (prototype) rather than creating from scratch. It allows you to clone objects, enabling fast object creation when instantiation is expensive or complex. It's based on the principle of cloning an existing object to get a new instance efficiently.

Why Use Prototype Pattern?

- **Performance**: Faster than creating objects from scratch (especially for complex objects)
- **Deep Cloning**: Create independent copies without inheritance complications
- **Lazy Initialization**: Defer expensive initialization to later cloning
- **Circular Reference Handling**: Clone objects with complex relationships
- **Reduce Subclassing**: Avoid creating multiple subclasses for variations
- **Dynamic Object Cloning**: Create objects dynamically at runtime
- **System Independence**: Clone objects without knowing their concrete classes
- **State Snapshots**: Easy checkpoint/rollback mechanisms

Key Components

Component	Responsibility
Prototype Interface	Declares clone method (e.g., Cloneable)
ConcretePrototype	Implements clone method to create copies
Client	Creates objects by cloning existing prototypes
PrototypeRegistry (Optional)	Manages and provides prototypes

Implementation

1. Basic Prototype Pattern (Shallow Copy)

Prototype Interface:

```
// Using Java's Cloneable interface
interface GamePiece extends Cloneable {
  GamePiece clone();
  void display();
```

```
}
```

ConcretePrototype - GamePiece.java:

```
class GamePiece implements Cloneable {
  private String color;
  private int id;

  public GamePiece(String color, int id) {
    this.color = color;
    this.id = id;
  }

  @Override
  public GamePiece clone() {
    try {
      return (GamePiece) super.clone();  // Shallow copy
    } catch (CloneNotSupportedException e) {
      throw new RuntimeException("Clone not supported", e);
    }
  }

  @Override
  public String toString() {
    return "GamePiece{" +
      "color='" + color + '\'' +
      ", id=" + id +
      '}';
  }

  public void setColor(String color) {
    this.color = color;
  }

  public void setId(int id) {
    this.id = id;
  }
}
```

ConcretePrototype - GameBoard.java:

```
class GameBoard implements Cloneable {
  private List<GamePiece> pieces = new ArrayList<>();

  public void addPiece(GamePiece piece) {
    pieces.add(piece);
  }

  @Override
```

```
  public GameBoard clone() {
    try {
      GameBoard cloned = (GameBoard) super.clone();
      cloned.pieces = new ArrayList<>(this.pieces);  // Deep copy list
      return cloned;
    } catch (CloneNotSupportedException e) {
      throw new RuntimeException("Clone failed", e);
    }
  }

  public void showBoardState() {
    System.out.println("Board State:");
    pieces.forEach(System.out::println);
  }
}
```

Key Points:

- Implement `Cloneable` interface
- Override `clone()` method using `super.clone()`
- Handle `CloneNotSupportedException`
- Shallow copy - only copies primitive values and object references
- For complex objects, implement deep copy in clone method

Prototype Variants

Approach 1: Shallow Copy (Shown Above)

- Pros: Simple, fast, minimal overhead
- Cons: Shared mutable objects between original and clone
- Best for: Objects with primitive fields or immutable objects

Approach 2: Deep Copy

```
@Override
public Employee clone() {
  try {
    Employee cloned = (Employee) super.clone();
    cloned.address = new Address(address.getCity(), address.getCountry());
    cloned.projects = new ArrayList<>(this.projects);
    return cloned;
  } catch (CloneNotSupportedException e) {
    throw new RuntimeException("Clone failed", e);
  }
}
```

Approach 3: Prototype Registry (Object Pool)

```
class ShapeRegistry {
```

```java
  private Map<String, Shape> shapes = new HashMap<>();

  public void addShape(String name, Shape shape) {
    shapes.put(name, shape);
  }

  public Shape getShape(String name) {
    Shape shape = shapes.get(name);
    return shape != null ? shape.clone() : null;
  }
}
```

Approach 4: Copy Constructor Alternative

```java
public Document(Document original) {
  this.title = original.title;
  this.content = new ArrayList<>(original.content);
  this.metadata = new HashMap<>(original.metadata);
}
```

Usage Example

GameClientWithPrototypePattern.java - Demonstrates prototype pattern:

```java
public class GameClientWithPrototypePattern {
  public static void main(String[] args) {

    // Create original board
    GameBoard gameBoard = new GameBoard();
    gameBoard.addPiece(new GamePiece("Red", 1));
    gameBoard.addPiece(new GamePiece("Blue", 5));
    gameBoard.showBoardState();

    // CheckPoint state: Clone for backup
    GameBoard copiedBoard = gameBoard.clone();

    System.out.println("\nCopy Game Board State:");
    copiedBoard.showBoardState();

    // Modify original without affecting clone
    gameBoard.addPiece(new GamePiece("Green", 10));

    System.out.println("\nOriginal after modification:");
    gameBoard.showBoardState();

    System.out.println("\nClone (unchanged):");
    copiedBoard.showBoardState();
  }
}
```

Output

```
Board State:
GamePiece{color='Red', id=1}
GamePiece{color='Blue', id=5}
```

```
Copy Game Board State:
GamePiece{color='Red', id=1}
GamePiece{color='Blue', id=5}

Original after modification:
GamePiece{color='Red', id=1}
GamePiece{color='Blue', id=5}
GamePiece{color='Green', id=10}

Clone (unchanged):
GamePiece{color='Red', id=1}
GamePiece{color='Blue', id=5}
```

Code Flow

1. Prototype object is created (expensive operation)
2. When clone needed, client calls `clone()` method
3. `super.clone()` creates new instance with copied state
4. Shallow copy: primitives copied, references shared
5. Deep copy: nested objects also cloned
6. Client receives independent copy
7. Modifications to clone don't affect original

Shallow Copy vs Deep Copy

Aspect	Shallow Copy	Deep Copy
Primitive Fields	Copied as new values	Copied as new values
Object References	Reference same object	Separate new objects
Nested Objects	Shared between copies	Independent copies
Modifications	Affect both original and clone	Only affects clone
Performance	Very fast	Slower (more work)
Complexity	Simple implementation	Complex implementation
Memory	Lower usage	Higher usage
Best For	Immutable/simple objects	Complex nested objects

Prototype vs Factory Pattern

Aspect	Prototype	Factory
Object Creation	Clone existing object	Create new from scratch
Existing Object	Required (template)	Not needed

Aspect	Prototype	Factory
Cost	Fast (copy)	Variable
Object Variation	Modify clone after creation	Use inheritance/parameters
Registration	Often uses registry	Not typically
Flexibility	Add objects at runtime	Fixed at compile-time
Dependency	Requires prototype instance	Only needs type info
Performance	Excellent (pre-initialized)	Good (creation overhead)

Real-World Analogies

- **Photo Copying**: Original document → Photocopy (Xerox)
- **DNA Cloning**: Template DNA → Clone organism
- **Blueprint Replication**: Master plans → Copies for buildings
- **Email Templates**: Template email → Clone and customize
- **Database Snapshots**: Backup database → Clone for testing
- **Computer Image**: Master installation → Cloned to multiple machines
- **Video Streaming**: Master content → Cloned across servers
- Cloning large objects where construction is costly (e.g., deep configuration settings).
- When system resources (time, memory) are limited and re-creating objects is expensive.
- Graphics objects cloning (shapes, images, drawings)
- Game objects duplication (enemies, power-ups)
- Configuration cloning with modifications
- UI component cloning
- Data entity cloning for versioning
- Cache implementations
- Undo/Redo functionality
- Database connection pool cloning
- Thread pool creation from prototype

Important Considerations

Implementing Cloneable

- `Cloneable` is a marker interface (no methods to implement)
- Must explicitly override `clone()` method
- Must handle `CloneNotSupportedException`
- `super.clone()` performs shallow copy

Deep vs Shallow Copy Decision

- Choose based on object complexity
- Shallow copy easier, but shared mutable objects risky
- Deep copy safer but more complex implementation
- Hybrid approach: partial deep copy for critical fields

Thread Safety

- Cloning itself is not inherently thread-safe
- Prototype registry should be synchronized for multi-threading
- Clone operations generally thread-safe if deep copy done properly
- Consider synchronization when cloning shared prototypes

Clone Method Conventions

- Always override `clone()` even if interface doesn't require it
- Return type should match actual class (covariant return type)
- Document shallow vs deep copy behavior
- Ensure copied object is completely independent

---

4. Factory Design Pattern

Consider an example of a transportation service app where users can request different types of transport vehicles (e.g., Car, Bike, Bus). You might initially create separate classes for each type, and create instances like this:

```
Car car = new Car();
Bike bike = new Bike();
```

But as the system evolves, managing object creation directly like this can become complex, especially when adding new types of vehicles.**Code**

Problems:

The client code (i.e., TransportService) is tightly coupled to concrete classes (Car, Bike, Bus).

Adding new transport types requires modifying client code.

The **Factory Pattern** helps centralize the creation logic and delegates the responsibility of creating objects to factory classes, which decide the specific class to instantiate. This allows the code to adhere to the **Open/Closed Principle** by letting new types of vehicles be added without modifying the existing code.

Factory Class: The TransportFactoryclass contains the logic to create different types of transport based on the input string. This abstracts the creation logic and makes it easier to add or change transport types.

Decoupling: The TransportServiceclass (client) no longer needs to know the details of how Car, Bike, or Busare created. It simply calls the factory method.

Flexibility: Adding a new transport type (e.g., Truck) only requires modifying the factory, not the client code.

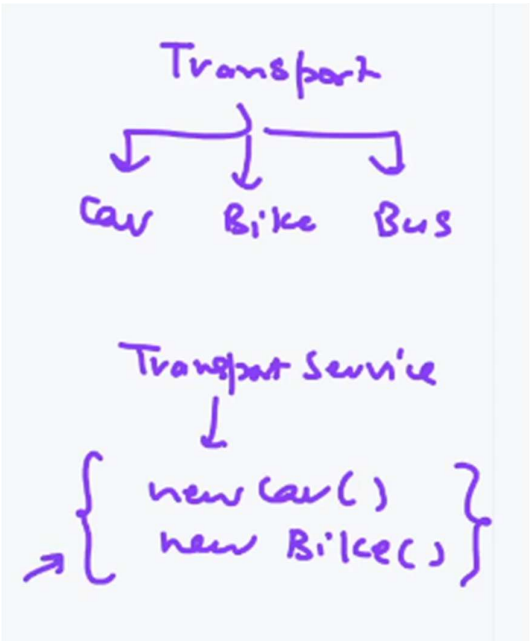
Factory Design Benefits

Benefits of Factory Pattern:

Loose Coupling: The client is decoupled from the specifics of object creation.

Single Responsibility Principle: The factory class handles the responsibility of object creation.

Open/Closed Principle: We can easily add new transport types without changing the client code, making the system open to extension and closed to modification.



Factory Design Pattern

The Factory Design Pattern is a **Creational Design Pattern** that provides an interface for creating objects without specifying the exact classes to create. Instead of directly instantiating objects using the `new` keyword, we use a factory method or factory class to create objects.

Types of Factory Patterns

1. Simple Factory Pattern

A single factory class that creates objects based on input parameters.

Characteristics:

- Not considered a true design pattern (more of a programming idiom)
- Uses a switch/if-else statement to determine object type
- Simple and straightforward implementation
- Violates Open/Closed Principle (OCP)

Example:

```
class ShapeFactory {
    public static Shape createShape(String type) {
        if (type.equals("CIRCLE")) {
            return new Circle();
        } else if (type.equals("RECTANGLE")) {
            return new Rectangle();
        }
        return null;
    }
}
```

---

2. Factory Method Pattern (Virtual Constructor Pattern)

Uses abstract methods in a base class to define the interface for creating objects. Subclasses override these methods to create specific object types.

Characteristics:

- Considered a true design pattern (part of Gang of Four patterns)
- Uses inheritance and method overriding
- Follows the Open/Closed Principle
- Each subclass is responsible for creating its own products

Key Components:

- **Creator (Abstract Class):** Declares the factory method
- **ConcreteCreator:** Implements the factory method
- **Product (Interface/Abstract):** Defines the interface for objects
- **ConcreteProduct:** Implements the product interface

Benefits:

- Eliminates dependency on concrete classes
- Easy to extend with new product types
- Follows Single Responsibility Principle (SRP)
- Leverages polymorphism

Example:

```
abstract class DatabaseConnection {
    abstract Connection createConnection();

    public void connectToDatabase() {
        Connection conn = createConnection();
        conn.connect();
    }
}

class MySQLConnection extends DatabaseConnection {
    @Override
    Connection createConnection() {
        return new MySQLConnectionImpl();
    }
}
```

When to Use Factory Patterns

Use Simple Factory when:

- You have a few, unchanging object types
- The creation logic is simple
- Extensibility is not a concern

Use Factory Method when:

- You want subclasses to decide which class to instantiate
- The number of product types may increase in future
- You want to avoid tight coupling to concrete classes

Real-World Examples

Example 1: Database Connection Factory

```
interface Database {
    void connect();
}
```

```java
class MySQLDatabase implements Database {
    public void connect() {
        System.out.println("Connecting to MySQL");
    }
}

class PostgreSQLDatabase implements Database {
    public void connect() {
        System.out.println("Connecting to PostgreSQL");
    }
}

class DatabaseFactory {
    public static Database createDatabase(String type) {
        if (type.equals("MYSQL")) return new MySQLDatabase();
        if (type.equals("POSTGRESQL")) return new PostgreSQLDatabase();
        return null;
    }
}
```

Example 2: Transport Delivery System (Similar to Bike.java)

```java
interface Transport {
    void deliver();
}

class Bike implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivering by bike");
    }
}

class Truck implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivering by truck");
    }
}

class Plane implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivering by plane");
    }
}

class TransportFactory {
    public static Transport createTransport(String type) {
        if (type.equals("BIKE")) return new Bike();
        if (type.equals("TRUCK")) return new Truck();
        if (type.equals("PLANE")) return new Plane();
        return null;
    }
```

```
}
```

Best Practices

1. **Use Interfaces:** Define contracts for factories and products using interfaces
2. **Single Responsibility:** Each factory should be responsible for one family of products
3. **Naming Convention:** Use clear names like `Factory`, `Creator`, or `Builder` in class names
4. **Configuration:** Use configuration files or properties to specify factory implementation
5. **Registry Pattern:** Combine with Registry pattern for more flexible object creation
6. **Dependency Injection:** Consider using DI frameworks for automatic factory selection

Real World Use Cases

GUI Frameworks: When the type of button or widget to be created is determined at runtime based on the platform (e.g., Windows, macOS, Linux).

Database Connectivity: When choosing different types of databases (e.g., SQL, NoSQL) based on configuration.

Document Conversion Tools: Where the type of file (e.g., PDF, Word, HTML) to be created depends on user input or settings.

---

5. Abstract Factory Pattern

Consider an application that needs to support multiple UI themes (e.g., Windows, macOS). Each theme has its own set of UI components such as buttons, scrollbars, and windows. The challenge is to create an architecture that allows switching between these themes without changing the client code that uses the UI components.

Without the Abstract Factory Pattern, the client code would be tightly coupled with the concrete implementations of buttons, scrollbars, etc., and switching between themes would require modifying the client code.

Problems in Code

Tight coupling: The client code is tightly coupled to specific implementations of UI components (WindowsButton, WindowsScrollBar).

Hard to extend: If we want to add support for macOS UI components, we would need to modify the client code to create instances of MacOSButtonand MacOSScrollBar.

Abstract Factory Pattern

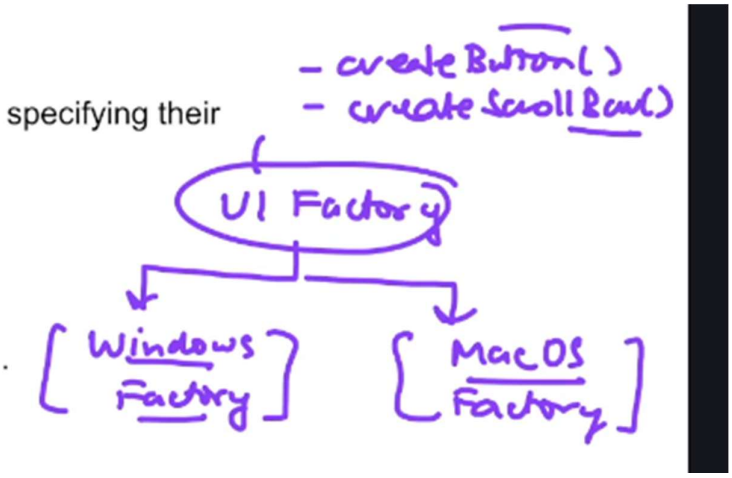
Problem It Solves: Provides an interface for creating **families of related objects** without specifying their concrete classes.

Structure:

Abstract Factory: Interface for creating abstract products.

Concrete Factory: Implements the abstract factory and creates concrete products.

Using the Abstract Factory Pattern, you create interfaces for each product (e.g., Button, ScrollBar, Window) and provide a family of concrete implementations for each theme (e.g., WindowsButton, MacOSButton, etc.). The Abstract Factory provides a way to create a suite of related objects without knowing the exact type of objects that will be created.



Factory vs Abstract Factory

Factory Method: Defines a method in the base class but lets subclasses override it to specify the type of objects that will be created.

Example: A DocumentFactorysubclass that creates either PDF or Word documents.

Abstract Factory: Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

Example: A GUIFactorythat creates related objects like buttons, checkboxes, and text fields depending on the operating system (Windows or Mac).

Abstract Factory Design Pattern

The Abstract Factory Pattern is a **Creational Design Pattern** that provides an interface for creating **families of related or dependent objects** without specifying their concrete classes. It's used when systems need to work with multiple families of products independently.

Key Components

1. **AbstractFactory:** Interface defining methods to create abstract products
2. **ConcreteFactory:** Implements factory methods for specific product families
3. **AbstractProduct:** Interface for product families
4. **ConcreteProduct:** Specific product implementations

Code Example (Under 50 Lines)

```java
// Product Interfaces
interface Button { void render(); }
interface ScrollBar { void scroll(); }

// Concrete Products
class WindowsButton implements Button {
  public void render() { System.out.println("Windows Button"); }
}
class WindowsScrollBar implements ScrollBar {
  public void scroll() { System.out.println("Windows ScrollBar"); }
}
class MacButton implements Button {
  public void render() { System.out.println("Mac Button"); }
}
class MacScrollBar implements ScrollBar {
  public void scroll() { System.out.println("Mac ScrollBar"); }
}

// Abstract Factory
interface UIFactory {
  Button createButton();
  ScrollBar createScrollBar();
}

// Concrete Factories
class WindowsFactory implements UIFactory {
  public Button createButton() { return new WindowsButton(); }
  public ScrollBar createScrollBar() { return new WindowsScrollBar(); }
}
class MacFactory implements UIFactory {
  public Button createButton() { return new MacButton(); }
  public ScrollBar createScrollBar() { return new MacScrollBar(); }
}

// Client Code
class Application {
  UIFactory factory;
  Application(UIFactory factory) { this.factory = factory; }
  void renderUI() {
    factory.createButton().render();
    factory.createScrollBar().scroll();
  }
}
```

Problem It Solves

Before (Tight Coupling):

```
if (OS.equals("Windows")) {
  button = new WindowsButton();
  scrollBar = new WindowsScrollBar();
} else if (OS.equals("Mac")) {
  button = new MacButton();
  scrollBar = new MacScrollBar();
}
```

❌ Violates Open/Closed Principle ❌ Hard to maintain and extend

After (Abstract Factory):

```
UIFactory factory = OSFactory.getFactory(OS);
button = factory.createButton();
scrollBar = factory.createScrollBar();
```

✅ Clean and maintainable ✅ Easy to add new OS families

Benefits

✅ **Loose Coupling:** Isolates concrete classes ✅ **Consistency:** Ensures related objects work together ✅ **Easy Extension:** Add new product families easily ✅ **Centralized:** All creation logic in factory classes ✅ **SOLID:** Follows Open/Closed and Dependency Inversion

Comparison Table: Factory Method vs Abstract Factory

Feature	Factory Method	Abstract Factory
Purpose	Create single product objects	Create families of related objects
Interface	Single factory method	Multiple factory methods
Products	One type of product	Multiple related product types
Extensibility	Add new products via subclasses	Add new product families via new factories
Coupling	Moderate	Low
Complexity	Low-Medium	Medium-High
Use Case	Single object creation	Coordinated multi-object creation
Example	ShapeFactory.create(type)	UIFactory.createButton() & .createScrollBar()
Inheritance	Typically uses inheritance	Uses composition/interfaces
Product Consistency	Not enforced	Enforced (same family)

Real-World Applications

- **UI Frameworks:** Buttons, scrollbars, menus for different OS
- **Database Systems:** Connection objects, statement objects for different DBs
- **Document Processing:** Word, Excel, PDF document families
- **Game Development:** Character, weapon, and item families for different game themes
- **Theme Systems:** Dark mode, light mode, high-contrast mode families

---

Comparison: Abstract Factory vs Simple Factory

Aspect	Simple Factory	Abstract Factory
Object Creation	Single method	Multiple methods
Families	None	Multiple
Design Pattern	No (Idiom)	Yes (GoF)
Coupling	Medium	Low
Extensibility	Poor	Excellent
Code Size	Small	Large

Module 3.3: Creational Patterns

- Overview of Structural Design Patterns

1. Adapter Pattern
2. Decorator Pattern
3. Proxy Pattern
4. Composite Pattern
5. Facade Pattern
6. Bridge Pattern
7. Flyweight Patten

Overview of Structural Design Patterns

Structural patterns are design patterns that deal with how objects and classes are **composed** to form larger structures, making the system more flexible and easy to understand.

They simplify the design of complex systems by creating **relationships**

between objects.

They help in **organizing class hierarchies** and allow for easy modification and extension.

---

1. Adapter Design Pattern

Adapter Pattern

The **Adapter Pattern** is a structural design pattern that allows objects with incompatible interfaces to work together.

Problem Statement

Imagine you are working on an **e-commerce application** that needs to send **email notifications** to customers. Currently, you are using your own in-house EmailNotificationService, but now you want to integrate a popular third-party email service like **SendGrid**. However, the interface for your in-house system and the SendGrid service are incompatible.

Let's see how the **Adapter Pattern** can solve this problem.

Adapter Pattern

Problem: When two systems or components have incompatible interfaces, they cannot work together directly.

Solution: The Adapter Pattern bridges the gap by **converting the interface** of one class into another that the client expects.

Real-world analogy**:**

A **power adapter** that allows a device with a US power plug to fit and work with a European power socket

Adapters in Software Frameworks: In GUI frameworks, adapters are used to convert legacy code into newer formats.

Adapter in Java I/O: In Java, InputStreamReaderworks as an adapter to convert InputStream(byte-based) to Reader(character-based).

Adapter in APIs: When integrating external libraries, you often need adapters to convert data formats or APIs to match your system's requirements.

The Adapter Pattern is a versatile solution to ensure incompatible interfaces work together seamlessly, making it invaluable in software integration and legacy system migration.

Adapter Pattern Benefits

Reusability: You can reuse existing code even when its interface is not compatible with the current system.

Flexibility: It helps integrate classes from different libraries or systems that were not designed to work together.

Decoupling: The Adapter decouples the client from the specific implementation of the Adaptee.

Adapter Design Pattern

The **Adapter Pattern** is a structural design pattern that allows incompatible interfaces to work together by wrapping an existing class with a new interface. It acts as a bridge between two incompatible classes/interfaces, enabling them to collaborate without modifying their source code. This pattern is particularly useful when integrating third-party libraries or legacy code.

Why Use Adapter Pattern?

- **Legacy Code Integration**: Use old code with new interfaces without modification
- **Third-Party Integration**: Adapt third-party libraries to your system's interface
- **Interface Mismatch**: Bridge incompatible interfaces between components
- **Code Reusability**: Reuse existing classes even if interfaces don't match
- **Reduce Code Duplication**: Avoid rewriting existing functionality
- **Single Responsibility**: Keep classes focused on their core responsibility
- **Open/Closed Principle**: Add new functionality without modifying existing code
- **Loose Coupling**: Decouple client code from specific implementations

Key Components

Component	Responsibility
Target Interface	Interface expected by the client
Adaptee/Adapter Source	Existing class with incompatible interface
Adapter	Converts adaptee interface to target interface
Client	Uses target interface without knowing about adaptee

Two Types of Adapter Pattern

Type 1: Class Adapter (Using Inheritance)

- Uses multiple inheritance (Java doesn't support, use interfaces)
- Adapter inherits from both target and adaptee
- Inflexible, rarely used in Java

Type 2: Object Adapter (Using Composition)

- Adapter wraps adaptee object
- More flexible and commonly used
- Preferred approach in modern development

Implementation

1. Object Adapter Pattern (Recommended)

Target Interface - NotificationService.java:

```
// This is what the client expects
public interface NotificationService {
  void send(String to, String subject, String body);
}
```

Adaptee - SendGridService.java:

```
// Third-party service with incompatible interface
public class SendGridService {
  // This method has different signature/behavior
  public void sendEmail(String recipient, String title, String message) {
    System.out.println("SendGrid: Sending email");
    System.out.println("To: " + recipient);
    System.out.println("Title: " + title);
    System.out.println("Message: " + message);
  }
}
```

Existing Target Implementation - EmailNotificationService.java:

```
// Current implementation using target interface
public class EmailNotificationService implements NotificationService {
  @Override
  public void send(String to, String subject, String body) {
    System.out.println("Email: Sending notification");
    System.out.println("To: " + to);
    System.out.println("Subject: " + subject);
    System.out.println("Body: " + body);
```

```
  }
}
```

Adapter - SendGridAdapter.java:

```
// Adapter that wraps SendGridService and implements NotificationService
public class SendGridAdapter implements NotificationService {
  // Compose the adaptee
  private SendGridService sendGridService;

  // Accept adaptee in constructor
  public SendGridAdapter(SendGridService sendGridService) {
    this.sendGridService = sendGridService;
  }

  // Implement target interface by delegating to adaptee
  @Override
  public void send(String to, String subject, String body) {
    // Convert interface (map method parameters)
    sendGridService.sendEmail(to, subject, body);
  }
}
```

Key Points:

- Adapter implements target interface
- Adapter wraps adaptee object (composition)
- Adapter maps target methods to adaptee methods
- Client uses adapter through target interface
- Adaptee interface remains unchanged

Adapter Variants

Approach 1: Object Adapter (Shown Above)

- Pros: Flexible, uses composition, avoids multiple inheritance issues
- Cons: Wrapper adds slight overhead
- Best for: Wrapping incompatible classes

Approach 2: Class Adapter (Inheritance)

```
// Java doesn't support multiple inheritance, but interface approach:
public class PayPalAdapter extends PayPalSDK implements PaymentService {
  @Override
  public void pay(double amount, String currency) {
    executePayment(amount, currency);
  }
}
```

Approach 3: Two-Way Adapter (Bidirectional)

```
public class BidirectionalAdapter implements OldInterface, NewInterface {
  private LegacySystem legacy;
  private ModernSystem modern;

  public BidirectionalAdapter(LegacySystem legacy, ModernSystem modern) {
    this.legacy = legacy;
    this.modern = modern;
  }

  // Implement both interfaces, converting between them
}
```

Approach 4: Adapter Chain (Multiple Adapters)

```
// Stack multiple adapters
NotificationService adapter1 = new EmailAdapter(emailService);
NotificationService adapter2 = new EncryptionAdapter(adapter1);
NotificationService adapter3 = new LoggingAdapter(adapter2);
adapter3.send(to, subject, body);
```

Usage Example

Client.java - Demonstrates adapter pattern:

```
public class Client {
  public static void main(String[] args) {

    // Using compatible email service directly
    NotificationService emailService = new EmailNotificationService();
    emailService.send(
      "customer@ab.com",
      "Your order has been shipped!",
      "Order details are ...."
    );

    System.out.println("------------------------------");

    // Using third-party SendGridService through adapter
    // SendGridService doesn't implement NotificationService
    // But SendGridAdapter bridges the gap
    NotificationService sendGridAdapter =
      new SendGridAdapter(new SendGridService());

    sendGridAdapter.send(
      "customer@ab.com",
      "Your order has been shipped!",
      "Order details are ...."
    );
  }
}
```

Output

```
Email: Sending notification
To: customer@ab.com
Subject: Your order has been shipped!
Body: Order details are ....
-----------------------------
SendGrid: Sending email
To: customer@ab.com
Title: Your order has been shipped!
Message: Order details are ....
```

Code Flow

1. Client calls method on target interface
2. Adapter receives the call
3. Adapter converts parameters if needed
4. Adapter delegates to adaptee method
5. Adaptee executes original functionality
6. Result returned through adapter to client
7. Client receives result in expected format

Real-World Examples

Scenario	Adaptee	Target	Adapter
SendGrid Integration	SendGridService	NotificationService	SendGridAdapter
PayPal Payment	PayPalSDK	PaymentProcessor	PayPalAdapter
Database Driver	LegacyOracleDriver	JDBCDriver	OracleJDBCAdapter
Logging Framework	Log4jLogger	SLF4JLogger	Log4jSLF4JAdapter
HTTP Client	OkHttpClient	HttpUrlConnection	OkHttpAdapter
Graphics Rendering	OpenGLRenderer	DirectXInterface	OpenGLDirectXAdapter
Date Format	LegacyDateFormat	JavaTimeFormat	DateFormatAdapter
Payment Gateway	StripeAPI	PaymentGateway	StripeAdapter

Real-World Analogy

Power Adapter:

- **Target Interface**: International power outlet (what device expects)
- **Adaptee**: Country-specific power outlet (different standard)
- **Adapter**: Physical power converter (bridges the gap)
- **Client**: Electrical device (works with adapter)
- Integrating third-party services (payment gateways, email providers)
- Using legacy database drivers with new code
- Connecting different logging frameworks
- Adapting UI components from different libraries
- Converting data formats between systems
- Using incompatible API responses
- Integrating hardware interfaces
- Converting between file formats

---

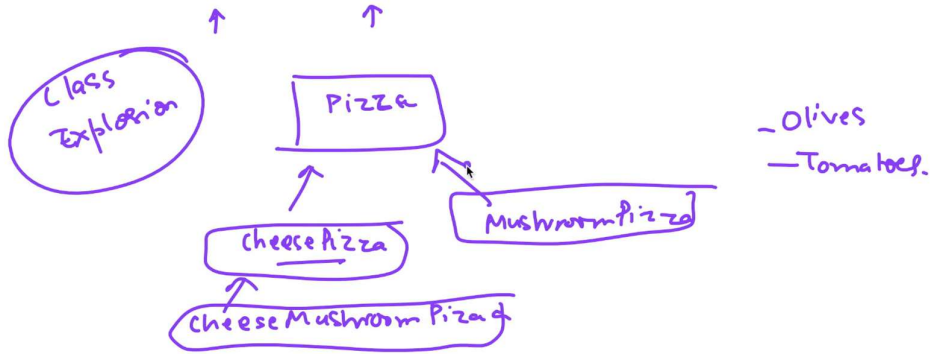
2. Decorator Pattern

You need to add functionality to an object at runtime, but subclassing would lead to an explosion of subclasses or is impractical.

Let's say we have a simple Pizza ordering system. Initially, we just have plain pizza but now we want to add options such as cheese, olives, tomatoes and mushrooms without modifying the existing pizza class or creating multiple subclasses.

Let's use the **Decorator Pattern** using a Pizza making example where different toppings (like cheese, olives, tomatoes, etc.) can be dynamically added to the base pizza.

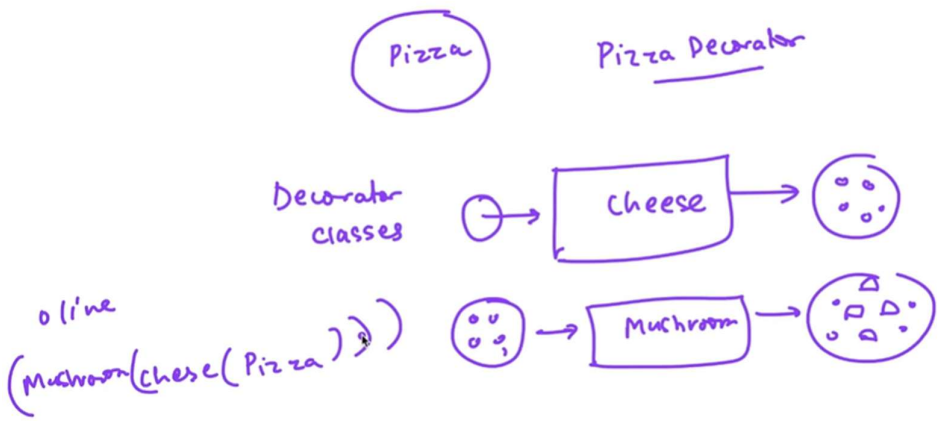
Problem: In a system where you need to **dynamically extend or add behavior** to objects, inheritance can lead to inflexible and tightly coupled code.



Solution: The Decorator Pattern allows you to **add new functionality to objects at runtime** by wrapping them with decorator classes, providing flexibility

Real-world analogy**:**

A **pizza order** where you start with a basic pizza and add toppings dynamically (e.g., cheese, pepperoni) without modifying the original pizza class.



lexible and Scalable: You can add as many toppings as needed by simply creating a decorator for each topping. No need for subclassing every combination.

Single Responsibility Principle: Each decorator class has one responsibility — to add a specific topping.

Open/Closed Principle: The BasicPizzaclass remains unchanged, and new features (toppings) can be added by creating new decorators.

Dynamically Changeable: Decorators can be added or removed dynamically at runtime.

Combinatorial Freedom: You can mix and match toppings in any order, making the system more flexible and reusable.

Decorator vs Inheritance

Inheritance is static and applied at compile-time, leading to tight coupling between base and derived classes.

Decorator Pattern provides a more flexible way to add or remove behavior at

runtime without changing the underlying object.

Decorator Design Pattern

The **Decorator Pattern** is a structural design pattern that allows you to attach additional responsibilities to an object dynamically, providing a flexible alternative to subclassing for extending functionality. It wraps an object with a decorator object that provides additional features while maintaining the same interface as the original object.

Why Use Decorator Pattern?

- **Dynamic Feature Addition**: Add features to objects at runtime without modifying source
- **Avoid Subclass Explosion**: Alternative to creating many subclasses for combinations
- **Single Responsibility**: Each decorator handles one specific enhancement
- **Flexible Composition**: Combine multiple decorators in any order
- **Open/Closed Principle**: Open for extension, closed for modification
- **Transparency**: Decorators are transparent to clients using the component
- **Runtime Behavior**: Change object behavior dynamically
- **Easy to Add/Remove**: Add or remove decorators without affecting other code

---

Key Components

Component	Responsibility
Component Interface	Defines interface for objects and decorators
ConcreteComponent	Base object that decorators wrap
Decorator	Abstract base for all decorators
ConcreteDecorator	Specific decorator that adds new behavior
Client	Works with decorated objects through component interface

---

Implementation

1. Decorator Pattern with Pizza Example

Component Interface - Pizza.java:

```
// Interface that both component and decorators implement
public interface Pizza {
  String getDescription();
  double getCost();
}
```

Concrete Component - BasicPizza.java:

```
// Base object that can be decorated
public class BasicPizza implements Pizza {
  @Override
```

```java
  public String getDescription() {
    return "Basic Pizza";
  }

  @Override
  public double getCost() {
    return 5.0;
  }
}
```

Abstract Decorator - PizzaDecorator.java:

```java
// Abstract base class for decorators
abstract class PizzaDecorator implements Pizza {
  // Holds reference to wrapped object
  protected Pizza decoratedPizza;

  // Accept wrapped object in constructor
  public PizzaDecorator(Pizza decoratedPizza) {
    this.decoratedPizza = decoratedPizza;
  }

  // Default implementation delegates to wrapped object
  @Override
  public String getDescription() {
    return decoratedPizza.getDescription();
  }

  @Override
  public double getCost() {
    return decoratedPizza.getCost();
  }
}
```

Concrete Decorators - CheeseDecorator.java:

```java
// Specific decorator that adds cheese
public class CheeseDecorator extends PizzaDecorator {
  public CheeseDecorator(Pizza pizza) {
    super(pizza);
  }

  @Override
  public String getDescription() {
    return decoratedPizza.getDescription() + ", cheese";
  }

  @Override
  public double getCost() {
    return decoratedPizza.getCost() + 1.5;
  }
}
```

More Concrete Decorators - PaneerDecorator.java:

```
public class PaneerDecorator extends PizzaDecorator {
  public PaneerDecorator(Pizza pizza) {
    super(pizza);
  }

  @Override
  public String getDescription() {
    return decoratedPizza.getDescription() + ", paneer";
  }

  @Override
  public double getCost() {
    return decoratedPizza.getCost() + 3.0;
  }
}
```

MushroomDecorator.java:

```
public class MushroomDecorator extends PizzaDecorator {
  public MushroomDecorator(Pizza pizza) {
    super(pizza);
  }

  @Override
  public String getDescription() {
    return decoratedPizza.getDescription() + ", mushrooms";
  }

  @Override
  public double getCost() {
    return decoratedPizza.getCost() + 2.0;
  }
}
```

Key Points:

- All decorators implement same interface as component
- Decorators wrap component (composition)
- Each decorator adds specific functionality
- Decorators can be stacked in any order
- Client sees single unified interface

Decorator Variants

Approach 1: Class Decorator (Shown Above)

- Pros: Simple inheritance hierarchy, clear structure

- Cons: Cannot decorate subclasses of wrapped object
- Best for: Decorating specific classes

Approach 2: Generic Decorator

```
public class GenericDecorator<T> implements Decorator<T> {
  private T component;
  private String decoration;

  public GenericDecorator(T component, String decoration) {
    this.component = component;
    this.decoration = decoration;
  }

  public T getComponent() { return component; }
  public String getDecoration() { return decoration; }
}
```

Approach 3: Stream Decorators (java.io)

```
// Reader stream wrapped with decorators
BufferedReader in = new BufferedReader(
  new InputStreamReader(
    new FileInputStream("file.txt")
  )
);
```

Approach 4: Functional Decorator (Lambda)

```
interface Decorator {
  String decorate(String input);
}

Decorator bold = s -> "<b>" + s + "</b>";
Decorator italic = s -> "<i>" + s + "</i>";
Decorator combined = bold.compose(italic);
```

Usage Example

PizzaApp.java - Demonstrates decorator pattern:

```
public class PizzaApp {
  public static void main(String[] args) {
    // Start with basic pizza
    Pizza pizza = new BasicPizza();
    System.out.println("Pizza: " + pizza.getDescription());
    System.out.println("Cost: $" + pizza.getCost());
    System.out.println();

    // Wrap with cheese decorator
    pizza = new CheeseDecorator(pizza);
    System.out.println("Pizza: " + pizza.getDescription());
```

```
    System.out.println("Cost: $" + pizza.getCost());
    System.out.println();

    // Wrap with paneer decorator
    pizza = new PaneerDecorator(pizza);
    System.out.println("Pizza: " + pizza.getDescription());
    System.out.println("Cost: $" + pizza.getCost());
    System.out.println();

    // Wrap with mushroom decorator
    pizza = new MushroomDecorator(pizza);
    System.out.println("Pizza: " + pizza.getDescription());
    System.out.println("Cost: $" + pizza.getCost());
  }
}
```

Output

```
Pizza: Basic Pizza
Cost: $5.0

Pizza: Basic Pizza, cheese
Cost: $6.5

Pizza: Basic Pizza, cheese, paneer
Cost: $9.5

Pizza: Basic Pizza, cheese, paneer, mushrooms
Cost: $11.5
```

Code Flow

1. Client instantiates base component (BasicPizza)
2. Client wraps component with first decorator (CheeseDecorator)
3. Decorator stores reference to wrapped object
4. Client can nest multiple decorators
5. Each level calls previous decorator's method
6. Behavior accumulates through decorator chain
7. Final object has all decorations applied

Decorator Chain Visualization

```
MushroomDecorator
   ↓
PaneerDecorator
   ↓
CheeseDecorator
   ↓
BasicPizza (5.0)
```

Decorator Chain Examples

Example 1: Simple Single Decorator

```
Pizza myPizza = new CheeseDecorator(new BasicPizza());
```

Example 2: Multiple Decorators Stacked

```
Pizza myPizza =
  new MushroomDecorator(
    new PaneerDecorator(
      new CheeseDecorator(
        new BasicPizza()
      )
    )
  );
```

Example 3: Dynamic Decorator Application

```
Pizza myPizza = new BasicPizza();
if (wantsCheese) myPizza = new CheeseDecorator(myPizza);
if (wantsPaneer) myPizza = new PaneerDecorator(myPizza);
if (wantsMushrooms) myPizza = new MushroomDecorator(myPizza);
```

Real-World Applications

Application	Base	Decorator 1	Decorator 2	Decorator 3
File I/O	FileInputStream	BufferedInputStream	DataInputStream	CompressedInputStream
UI Components	Panel	ScrollDecorator	BorderDecorator	ShadowDecorator
HTTP Requests	Request	AuthDecorator	CacheDecorator	LoggingDecorator
Pizza Order	BasicPizza	CheeseDecorator	PaneerDecorator	MushroomDecorator
Text Formatting	PlainText	BoldDecorator	ItalicDecorator	UnderlineDecorator
Database Queries	SQLQuery	CacheDecorator	LoggingDecorator	TimingDecorator
Stream Processing	InputStream	EncryptionDecorator	CompressionDecorator	ValidatingDecorator

Comparison: Inheritance vs Decorator

Using Inheritance (Bad for combinations):

```
class BasicPizza { }
class CheesePizza extends BasicPizza { }
class PaneerPizza extends BasicPizza { }
class CheesePaneerPizza extends CheesePizza { }
class MushroomPizza extends BasicPizza { }
class CheeseMushroomPizza extends CheesePizza { }
// ... explosion of classes!
```

Using Decorator (Flexible combinations):

```
Pizza pizza = new BasicPizza();
pizza = new CheeseDecorator(pizza);
pizza = new PaneerDecorator(pizza);
pizza = new MushroomDecorator(pizza);
// Clean and flexible!
```

Real-World Analogy

Gift Wrapping:

- **Base Object**: Gift (the actual item)
- **Decorators**: Gift wrap (basic), Ribbon (enhancement), Bow (another enhancement)
- **Combination**: You can apply wrap, ribbon, and bow in any order
- **Result**: Same gift, different visual appearance
- **Flexibility**: Wrap multiple times without creating new gift!.
- Adding UI features (scrollbars, borders, decorations)
- Extending component functionality (compression, encryption, caching)
- Pizza ordering system with toppings (your use case!)
- Stream wrapping (BufferedInputStream, DataInputStream)
- Logger enhancements (timestamps, formatting, severity levels)
- HTTP request/response interceptors
- Window manager effects (shadows, transparency)
- File I/O wrappers (encryption, compression, formatting)

---

3. Proxy Pattern

Problem: Sometimes, direct access to an object might not be desirable due to reasons such as security, resource optimization, or controlled access.

Solution: The Proxy Pattern provides a **surrogate** or placeholder for another object to control access to it.

Proxy Pattern Motivation

Imagine a system where we want to load a heavy object like a large image from disk. It might take time to load the image, and we don’t want to load it until it’s necessary. Without using a proxy, the application would load the image every time it's needed, even if not displayed, wasting resources.

Virtual Proxy: Delays the creation of expensive objects until they are needed (lazy initialization).

Protection Proxy: Controls access based on permissions.

Example

A **large image** loading in an application is expensive, so a virtual proxy is used to load the image only when it's actually needed (on-demand).

Proxy Pattern Benefits

Lazy Initialization: Objects are loaded only when necessary, saving memory and CPU cycles.

Access Control: You can control access to the real object (e.g., based on user permissions).

Additional Behavior: Proxies can add additional functionalities like logging, access control, or caching without modifying the real object.

Separation of Concerns: The real object only deals with its core responsibilities, while the proxy handles ancillary operations like initialization or security.

Proxy Design Pattern

The **Proxy Pattern** is a structural design pattern that provides a surrogate or placeholder for another object to control access to it. A proxy is an object that acts as an intermediary between the client and the real object, allowing you to control access, delay initialization, log calls, or add security checks before reaching the actual object.

Why Use Proxy Pattern?

- **Lazy Initialization**: Defer expensive object creation until actually needed
- **Access Control**: Restrict access to sensitive objects
- **Logging & Monitoring**: Log method calls and monitor usage
- **Caching**: Cache expensive computations or data
- **Security**: Add authentication/authorization before accessing real object
- **Remote Access**: Represent remote objects locally
- **Performance Optimization**: Reduce load by controlling when objects are created
- **Transparency**: Client unaware of proxy - sees uniform interface

Key Components

Component	Responsibility
Subject Interface	Defines common interface for real & proxy objects
RealSubject	Actual object that proxy represents
Proxy	Surrogate that wraps real subject
Client	Works with proxy through subject interface

Types of Proxy Pattern

1. Virtual Proxy (Lazy Initialization)

- Defers creation of expensive objects until needed
- Most common type
- Used in your image loading example

2. Remote Proxy

- Represents object in different address space
- Used for RMI, web services
- Handles network communication

3. Protection Proxy

- Controls access to real object
- Implements security/authorization
- Restricts who can access object

4. Smart Reference

- Provides additional functionality
- Logging, caching, reference counting
- Transparent to client

Implementation

1. Virtual Proxy Pattern (Lazy Initialization)

Subject Interface - Image.java:

```
// Interface that both real and proxy objects implement
public interface Image {
  void display();
}
```

Real Subject - RealImage.java:

```
// Actual object that performs expensive operations
public class RealImage implements Image {
```

```java
  private String fileName;

  public RealImage(String fileName) {
    this.fileName = fileName;
    loadImageFromDisk();  // Expensive operation
  }

  private void loadImageFromDisk() {
    System.out.println("Loading image from disk: " + fileName);
    try {
      Thread.sleep(2000);  // Simulate time-consuming loading
    } catch (InterruptedException e) {
      e.printStackTrace();
    }
  }

  @Override
  public void display() {
    System.out.println("Displaying image: " + fileName);
  }
}
```

Proxy - ProxyImage.java:

```java
// Proxy that controls access and defers real object creation
public class ProxyImage implements Image {
  private String fileName;
  private RealImage realImage;  // Holds reference to real object

  public ProxyImage(String fileName) {
    this.fileName = fileName;
    // Don't create real object yet - lazy initialization
  }

  @Override
  public void display() {
    // Create real object only when first needed
    if (realImage == null) {
      realImage = new RealImage(fileName);
    }
    // Delegate to real object
    realImage.display();
  }
}
```

Key Points:

- Proxy implements same interface as real object
- Proxy holds reference to real object
- Real object created only when first used
- Subsequent calls reuse cached real object
- Client unaware of proxy - sees uniform interface

Proxy Variants

Approach 1: Virtual Proxy (Shown Above)

- Pros: Defers expensive object creation, saves resources
- Cons: First call has initialization overhead
- Best for: Expensive objects that might not be used

Approach 2: Remote Proxy

```
public class RemoteImageProxy implements Image {
  private String imageUrl;

  public RemoteImageProxy(String imageUrl) {
    this.imageUrl = imageUrl;
  }

  @Override
  public void display() {
    // Fetch from remote server
    byte[] imageData = fetchFromServer(imageUrl);
    // Display locally
    displayImage(imageData);
  }
}
```

Approach 3: Protection Proxy

```
public class ProtectionProxy implements Image {
  private RealImage realImage;
  private User currentUser;

  public ProtectionProxy(RealImage realImage, User user) {
    this.realImage = realImage;
    this.currentUser = user;
  }

  @Override
  public void display() {
    if (currentUser.hasPermission("VIEW_IMAGE")) {
      realImage.display();
    } else {
      System.out.println("Access Denied!");
    }
  }
}
```

Approach 4: Caching Proxy

```
public class CachingProxy implements Image {
  private RealImage realImage;
  private static Map<String, RealImage> cache = new HashMap<>();

  public CachingProxy(String fileName) {
    if (!cache.containsKey(fileName)) {
      cache.put(fileName, new RealImage(fileName));
    }
    this.realImage = cache.get(fileName);
  }

  @Override
  public void display() {
    realImage.display();
  }
}
```

Usage Example

Client.java - Demonstrates proxy pattern:

```
public class Client {
  public static void main(String[] args) {

    // Create proxy objects (doesn't load images yet)
    Image image1 = new ProxyImage("photo1.jpg");
    Image image2 = new ProxyImage("photo2.jpg");

    System.out.println("Proxies created, but images not loaded");
    System.out.println();

    // First display - loads image (takes 2 seconds)
    System.out.println("First display call:");
    image1.display();
    System.out.println();

    // Second display - uses cached real image (instant)
    System.out.println("Second display call:");
    image1.display();
    System.out.println();

    // Different image - loads new image (takes 2 seconds)
    System.out.println("Display different image:");
    image2.display();
  }
}
```

Output

```
Proxies created, but images not loaded
```

```
First display call:
Loading image from disk: photo1.jpg
Displaying image: photo1.jpg

Second display call:
Displaying image: photo1.jpg

Display different image:
Loading image from disk: photo2.jpg
Displaying image: photo2.jpg
```

Code Flow

1. Client creates proxy instead of real object (lightweight)
2. Client calls method on proxy interface
3. Proxy checks if real object exists
4. If not, proxy creates real object (lazy initialization)
5. Proxy delegates method call to real object
6. Real object executes actual operation
7. Subsequent calls reuse existing real object
8. No initialization overhead on second call

Proxy Types Comparison

Type	Purpose	When to Use
Virtual	Lazy initialization	Heavy objects
Remote	Remote representation	Network objects
Protection	Access control	Secured objects
Smart Reference	Additional features	Logging, caching

Real-World Applications

Application	Real Object	Proxy	Purpose
Image Loading	RealImage	ProxyImage	Lazy load on demand
Database	DatabaseConnection	ConnectionPool	Control access, pool
Remote Server	ServerObject	RemoteProxy	Network access
File System	LargeFile	VirtualFile	Defer loading
Authentication	SecureResource	AuthProxy	Check permissions
Cache	ExpensiveQuery	CacheProxy	Store results
Web Service	RESTService	ServiceProxy	Monitor calls
Thread Pool	Worker	ThreadProxy	Manage execution

Proxy vs Lazy Initialization Pattern

Lazy Initialization without Proxy:

```
public class Service {
  private ExpensiveResource resource;

  public ExpensiveResource getResource() {
    if (resource == null) {
      resource = new ExpensiveResource();
    }
    return resource;
  }
}
```

Lazy Initialization with Proxy:

```
public class ServiceProxy extends Service {
  @Override
  public ExpensiveResource getResource() {
    if (super.getResource() == null) {
      return new ExpensiveResource();
    }
    return super.getResource();
  }
}
```

Real-World Analogy

Bank Account:

- **Real Subject**: Actual bank account with money
- **Proxy**: ATM card/interface
- **Access Control**: PIN authentication before withdrawals
- **Security**: Checks balance, limits, fraud detection
- **Result**: You don't handle real account directly, use proxy (card) instead!
- Image loading systems (lazy load images on demand)
- Database connection pools
- Remote Method Invocation (RMI)
- Web services and API calls
- Authentication/Authorization systems
- Caching systems (Redis, Memcached)
- Virtual proxies (defer expensive operations)
- Smart references (protection proxies)
- Logging and monitoring
- Transaction management

---

4. Composite Pattern

Problem: When building systems like a **file directory**, which consist of both individual items (files) and groups of items (directories), managing these with standard object hierarchies can become complex.

Solution: The Composite Pattern allows you to treat **individual objects** and

compositions of objects uniformly by representing part-whole hierarchies.

In a **file system**, both **File** and **Directory** can be treated as components. A directory can contain both files and other directories, forming a recursive tree structure.

Composite Design Pattern

The **Composite Pattern** is a structural design pattern that allows you to compose objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects and compositions of objects uniformly. This pattern is perfect for representing hierarchical structures like file systems, organizational charts, UI components, menus, and more.

Why Use Composite Pattern?

- **Tree Structures**: Naturally represent hierarchical structures
- **Uniform Interface**: Treat individual and composite objects the same way
- **Recursive Composition**: Easily build nested structures
- **Simplify Client Code**: No need to distinguish between leaf and composite
- **Flexibility**: Add new component types without changing existing code
- **Easy to Extend**: Add new operations to existing structure
- **Natural Hierarchy**: Reflects real-world hierarchies perfectly
- **Recursive Processing**: Automatically handle nested structures

Key Components

Component	Responsibility
Component Interface	Defines common interface for all objects
Leaf	Represents terminal objects with no children
Composite	Represents objects that can have children
Client	Works with objects through component interface

When to Use Composite vs Other Patterns

Composite is different from other structural patterns:

Aspect	Composite	Decorator	Facade
Purpose	Part-whole hierarchy	Add behavior	Simplify interface
Structure	Tree of objects	Single wrapper	Multiple wrappers
Children	Multiple	Single	Multiple
Interface	Uniform	Same	Different
Use Case	Hierarchies	Enhancement	Simplification

Implementation

1. Composite Pattern with File System

Component Interface - FileSystemComponent.java:

```
// Common interface for all components
public interface FileSystemComponent {
  void showDetails();
}
```

Leaf Component - File.java:

```
// Leaf object - cannot have children
public class File implements FileSystemComponent {
  private String name;

  public File(String name) {
    this.name = name;
  }

  @Override
  public void showDetails() {
    System.out.println("File: " + name);
  }
}
```

Composite Component - Folder.java:

```
// Composite object - can have children
public class Folder implements FileSystemComponent {
  private String name;
  private List<FileSystemComponent> components = new ArrayList<>();

  public Folder(String name) {
```

```
    this.name = name;
  }

  // Add method to add child components
  public void addComponent(FileSystemComponent component) {
    components.add(component);
  }

  // Remove method to remove child components
  public void removeComponent(FileSystemComponent component) {
    components.remove(component);
  }

  @Override
  public void showDetails() {
    System.out.println("Folder: " + name);
    // Recursively process all children
    for (FileSystemComponent component : components) {
      component.showDetails();
    }
  }
}
```

Key Points:

- Component interface defines common operations
- Leaf (File) implements interface but has no children
- Composite (Folder) implements interface and manages children
- Recursive processing through tree structure
- Client treats both the same way through interface

Composite Variants

Approach 1: Simple Composite (Shown Above)

- Pros: Simple, clear structure, easy to understand
- Cons: Operations for leaves and composites similar
- Best for: Basic hierarchies

Approach 2: Generic Composite

```
public abstract class Component<T> {
  protected String name;

  public abstract void operation();
}

public class Leaf<T> extends Component<T> {
```

```
  @Override
  public void operation() {
    // Leaf implementation
  }
}

public class Composite<T> extends Component<T> {
  private List<Component<T>> children = new ArrayList<>();
}
```

Approach 3: Safe Composite (Type Checking)

```
public interface Component {
  void operation();
  List<Component> getChildren();  // May throw exception for leaves
  void add(Component component);  // May throw exception for leaves
}
```

Approach 4: Transparent Composite (No Type Checking)

```
public abstract class Component {
  protected List<Component> children = new ArrayList<>();

  public void add(Component component) {
    children.add(component);
  }

  public abstract void operation();
}
```

Usage Example

FileSystemApp.java - Demonstrates composite pattern:

```
public class FileSystemApp {
  public static void main(String[] args) {

    // Create leaf components
    FileSystemComponent f1 = new File("file1.txt");
    FileSystemComponent f2 = new File("file2.txt");

    // Create composite (folder)
    Folder folder = new Folder("Documents");
    folder.addComponent(f1);
    folder.addComponent(f2);

    // Create nested subfolder
    Folder subFolder = new Folder("Subfolder");
    FileSystemComponent f3 = new File("file3.txt");
    subFolder.addComponent(f3);

    // Add subfolder to main folder
```

```
    folder.addComponent(subFolder);

    // Show entire tree structure
    folder.showDetails();
  }
}
```

Output

```
Folder: Documents
File: file1.txt
File: file2.txt
Folder: Subfolder
File: file3.txt
```

Code Flow

1. Create leaf components (files)
2. Create composite component (folder)
3. Add leaf components to composite
4. Create nested composite (subfolder)
5. Add nested composite to parent composite
6. Call operation on root composite
7. Operation recursively propagates through tree
8. Each node performs its own operation
9. Entire tree structure processed uniformly

Tree Structure Visualization

```
Folder: Documents
├── File: file1.txt
├── File: file2.txt
└── Folder: Subfolder
    └── File: file3.txt
```

Safe vs Transparent Composite

Safe Composite: Type Checking

```
public interface Component {
  void operation();
  // Only in Composite
  void add(Component component);
  void remove(Component component);
  Component getChild(int index);
```

```
}

public class Leaf implements Component {
  @Override
  public void add(Component component) {
    throw new UnsupportedOperationException();
  }
}
```

- Pros: Type safety, clear what operations are valid
- Cons: Client must check type before adding

Transparent Composite: No Type Checking

```
public interface Component {
  void operation();
  void add(Component component);
  void remove(Component component);
  Component getChild(int index);
}

public class Leaf implements Component {
  private List<Component> children = new ArrayList<>();

  @Override
  public void add(Component component) {
    children.add(component);  // Allowed but meaningless
  }
}
```

- Pros: Uniform interface, client doesn't check types
- Cons: Less type safe, all objects have all operations

Real-World Examples

File System

```
Drive: C:\
├── Folder: Documents
│   ├── File: resume.pdf
│   └── Folder: Projects
│       └── File: project.java
├── Folder: Pictures
│   ├── File: photo1.jpg
│   └── File: photo2.jpg
└── File: readme.txt
```

Organization Structure

```
Company
├── Department: Engineering
```

```
│   ├── Employee: John (Senior)
│   ├── Employee: Jane (Jr)
│   └── Team: Backend
│       ├── Employee: Bob
│       └── Employee: Alice
└── Department: Sales
    ├── Employee: Tom
    └── Employee: Jerry
```

UI Components

```
Frame
├── Panel (Main)
│   ├── Button (OK)
│   ├── Button (Cancel)
│   └── TextBox (Input)
└── Panel (Footer)
    ├── Label (Status)
    └── Button (Help)
```

Applications of Composite

Application	Leaf	Composite	Parent Container
File System	File	Folder	Drive/Root
Graphics	Shape	ShapeGroup	Canvas
UI	Button/Label	Panel	Window
Menu	MenuItem	Menu	MenuBar
Organization	Employee	Department	Company
Document	Paragraph	Section	Document
Game	Entity	EntityGroup	World
Expression	Operand	Operator	Expression

Composite vs Iterator Pattern

Composite: Represents structure

```
root.operation();  // Automatically handles tree
```

Iterator: Accesses elements sequentially

```
while (iterator.hasNext()) {
  element = iterator.next();  // Manual traversal
}
```

Both can be combined for flexible tree traversal!

Real-World Analogy

Organization Hierarchy:

- **CEO** (Root Composite) manages multiple departments
- **Department Manager** (Composite) manages multiple teams
- **Team Lead** (Composite) manages multiple employees
- **Individual Employee** (Leaf) performs work
- **Reports** (Operation) flow up the hierarchy
- **Decisions** (Operation) flow down the hierarchy
- File system representation (files and folders)
- UI component rendering (panels, buttons, layouts)
- Organizational hierarchies (employees, departments)
- Menu systems (menus with submenus)
- Document structures (chapters, sections, subsections)
- Graphics rendering (shapes, groups, layers)
- Expression trees (mathematical expressions)
- Game object hierarchies (game entities with children)

---

5. Facade Pattern

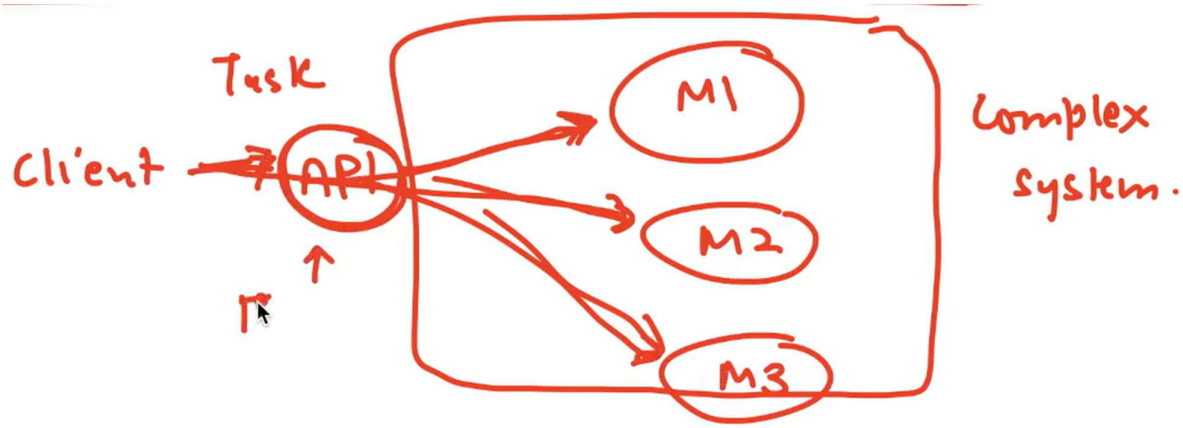
In software engineering, a common real-world example of the **Facade Pattern** is an **API Gateway** in a microservices architecture.

Problem Without Facade

In a microservices architecture, each microservice can have its own API for specific business logic, such as user management, order processing, and inventory. If the client needs to interact with these microservices, it would need to directly communicate with all the individual services. This would increase the complexity of the client code, tightly couple the client to all the microservices, and expose the inner workings of the system.

Solution Using Facade (API Gateway):

The **API Gateway** acts as a facade, providing a unified interface to the client while handling communication with the underlying microservices. It simplifies client interactions, reduces network calls, and abstracts away the complexity of dealing with multiple services.



Example: API Gateway as a Facade

In **microservices architectures**, an **API Gateway** acts as a facade, providing a simple interface to clients while hiding the complexity of multiple microservices working behind the scenes.

Problem: Large, complex subsystems with many classes and methods can be difficult to use directly.

Solution: The Facade Pattern provides a simple, unified interface to a complex subsystem, making it easier to interact with.

The **Facade Pattern** is a structural design pattern that provides a simplified interface to a complex system of classes, libraries, or frameworks. Instead of exposing all the details of the complex system, the facade offers a higher-level interface, making it easier to interact with the system. The Facade Pattern is particularly useful when dealing with large systems that contain many interdependent classes, by reducing the interaction points for the client.

Decoupling: The client is decoupled from the internal structure of the microservices. Any changes to the underlying services (e.g., changes in APIs or architecture) can be handled within the facade without impacting the client.

Reduced Complexity: The facade handles the orchestration of complex operations, hiding the complexity of multiple service interactions. This reduces the learning curve for developers working with the system.

Centralized Control: The facade (API Gateway in this case) can enforce security, logging, caching, and rate-limiting centrally. This avoids duplicating these concerns across multiple services or clients.

Easier Maintenance: With a facade in place, changes in one subsystem do not ripple out to clients. For example, if the order service API changes, the facade can adapt without needing to update the client code.

Enhanced Performance: By consolidating multiple requests into a single call through the facade, you can reduce network overhead, especially in distributed systems like microservices. For

example, fetching user details, order details, and payment information in a single request rather than three separate requests.

Consistency in Interfacing: The facade ensures a consistent interface for clients, even if the underlying systems evolve over time or become more complex. This helps keep the system modular and more understandable for future changes.

Facade Design Pattern

The **Facade Pattern** is a structural design pattern that provides a unified, simplified interface to a set of interfaces in a subsystem. It hides the complexity of internal subsystems and provides a single entry point for client interactions. The facade acts as a wrapper around multiple complex components, making them easier to use.

Why Use Facade Pattern?

- **Simplify Complex Systems**: Hide complexity behind a simple interface
- **Reduce Coupling**: Decouple clients from complex subsystems
- **Unified Access Point**: Single entry point for related functionality
- **Easier Maintenance**: Changes to subsystems don't affect clients
- **Layered Development**: Build systems in layers with clear boundaries
- **API Consistency**: Provide consistent interface across subsystems
- **Reduce Learning Curve**: Clients don't need to learn complex subsystem
- **Better Organization**: Organize related functionality logically

Key Components

Component	Responsibility
Facade	Provides simplified interface to subsystems
Subsystem Classes	Complex, independent implementation classes
Client	Uses facade instead of accessing subsystems directly

Implementation

1. Facade Pattern with E-Commerce Order System

Subsystem Classes:

UserService.java:

```
// Complex subsystem 1 - User management
public class UserService {
  public String getUserDetails(String userId) {
    // Simulate fetching user info from database
    return "User Info for " + userId;
  }
}
```

OrderService.java:

```
// Complex subsystem 2 - Order management
public class OrderService {
  public String getOrderDetails(String orderId) {
    // Simulate fetching order from database
    return "Order ID: " + orderId +
           ", Product: Laptop, Quantity: 1, Price: $1000";
  }
}
```

PaymentService.java:

```
// Complex subsystem 3 - Payment processing
public class PaymentService {
  public String processPayment(String paymentId) {
    // Simulate payment processing logic
    return "Payment with ID " + paymentId +
           " has been processed successfully.";
  }
}
```

Facade - APIGateway.java:

```
// Facade that simplifies interaction with multiple subsystems
public class APIGateway {
  // References to subsystems
  private UserService userService;
  private OrderService orderService;
  private PaymentService paymentService;

  // Constructor initializes all subsystems
  public APIGateway() {
    this.userService = new UserService();
    this.orderService = new OrderService();
    this.paymentService = new PaymentService();
  }
```

```java
// Single simplified method that coordinates multiple operations
public String getFullOrderDetails(String userId,
                                  String orderId,
                                  String amount) {
  // Facade orchestrates complex workflow
  String userDetails = userService.getUserDetails(userId);
  String orderDetails = orderService.getOrderDetails(orderId);
  String paymentStatus = paymentService.processPayment(amount);

  // Return unified result combining all subsystems
  return "User Details: " + userDetails + "\n" +
         "Order Details: " + orderDetails + "\n" +
         "Payment Status: " + paymentStatus;
}

// Other simplified facade methods
public String registerAndCheckout(String userId, String orderId) {
  String userDetails = userService.getUserDetails(userId);
  String orderDetails = orderService.getOrderDetails(orderId);
  return userDetails + " | " + orderDetails;
}
}
```

Key Points:

- Facade wraps multiple subsystems
- Facade provides simple unified interface
- Facade orchestrates complex operations
- Client only interacts with facade
- Subsystems remain independent and unchanged

Facade Variants

Approach 1: Simple Facade (Shown Above)

- Pros: Easy to implement, clear structure
- Cons: May become god object if too many responsibilities
- Best for: Small to medium subsystems

Approach 2: Layered Facade

```
// Layer 1: Service level
public class UserFacade {
  public String getUser(String userId) { }
}

// Layer 2: Business operation level
public class OrderFacade {
  private UserFacade userFacade;
  private OrderService orderService;

  public String processOrder(String userId, String orderId) {
    String user = userFacade.getUser(userId);
    String order = orderService.getOrder(orderId);
    return user + " | " + order;
  }
}

// Layer 3: API level (top-level facade)
public class APIGateway {
  private UserFacade userFacade;
  private OrderFacade orderFacade;

  public String getFullOrderDetails(String userId, String orderId) {
    return orderFacade.processOrder(userId, orderId);
  }
}
```

Approach 3: Generic Facade

```
public abstract class Facade {
  protected List<Subsystem> subsystems;

  public abstract void executeComplexOperation();
}

public class ConcreteFacade extends Facade {
  @Override
  public void executeComplexOperation() {
    // Coordinate all subsystems
  }
}
```

Approach 4: Builder Facade

```
public class APIGatewayBuilder {
  private UserService userService;
  private OrderService orderService;

  public APIGatewayBuilder withUserService(UserService service) {
    this.userService = service;
    return this;
  }
```

```
  public APIGatewayBuilder withOrderService(OrderService service) {
    this.orderService = service;
    return this;
  }

  public APIGateway build() {
    return new APIGateway(userService, orderService);
  }
}
```

Usage Example

Client.java - Demonstrates facade pattern:

```
public class Client {
  public static void main(String[] args) {

    // Without facade - client must deal with multiple services
    /*
    UserService userService = new UserService();
    OrderService orderService = new OrderService();
    PaymentService paymentService = new PaymentService();

    String user = userService.getUserDetails("U-007");
    String order = orderService.getOrderDetails("O-12345");
    String payment = paymentService.processPayment("P-123");

    System.out.println(user);
    System.out.println(order);
    System.out.println(payment);
    */

    // With facade - simple unified interface
    APIGateway apiGateway = new APIGateway();
    System.out.println(apiGateway.getFullOrderDetails("U-007", "O-12345", "P-123"));
  }
}
```

Output

```
User Details: User Info for U-007
Order Details: Order ID: O-12345, Product: Laptop, Quantity: 1, Price: $1000
Payment Status: Payment with ID P-123 has been processed successfully.
```

Code Flow

1. Client creates facade instance
2. Facade initializes all subsystems
3. Client calls single facade method
4. Facade coordinates multiple subsystem calls
5. Each subsystem performs its operation
6. Facade aggregates results
7. Client receives unified response
8. Complexity hidden from client

Interaction Diagram

```
Client
 ↓
APIGateway (Facade)
  ├→ UserService
  ├→ OrderService
  └→ PaymentService
```

Before and After Facade

Without Facade (Complex):

```
UserService userService = new UserService();
OrderService orderService = new OrderService();
PaymentService paymentService = new PaymentService();

String user = userService.getUserDetails("U-007");
String order = orderService.getOrderDetails("O-12345");
String payment = paymentService.processPayment("P-123");

System.out.println(user);
System.out.println(order);
System.out.println(payment);
// Client must know about all services and orchestrate
```

With Facade (Simple):

```
APIGateway apiGateway = new APIGateway();
System.out.println(apiGateway.getFullOrderDetails("U-007", "O-12345", "P-123"));
// Client only knows about facade, clean and simple
```

Real-World Applications

System	Subsystems	Facade	Benefit
E-Commerce	User, Order, Payment, Inventory	OrderGateway	Unified checkout
Banking	Account, Transfer, Loan	BankingFacade	Simple transactions
Database	Connection, Query, Transaction	ORM/DAL Layer	Simplified DB access
Compiler	Lexer, Parser, CodeGen, Optimizer	Compiler	Compile in one call
Web Framework	HTTP, Routing, Session, Cache	Framework	Simple request handling
Java IO	Files, Streams, Buffers	FileUtils	Easier file operations
Testing	Mocking, Assertions, Setup	TestFramework	Simple test writing
Graphics	Rendering, Lighting, Physics	GameEngine	Simple game development

---

Facade vs Similar Patterns

Pattern	When	Example
Facade	Simplify complex	APIGateway wrapping services
Adapter	Make incompatible compatible	Convert legacy interface
Proxy	Control access	Lazy load expensive object
Decorator	Add behavior	Wrap for enhancement
Bridge	Decouple abstraction	Abstract platform differences

Key Difference: Facade simplifies complexity, others have different purposes!

Layered Facades

Example: Database Access

```
Application Layer
   ↓
Facade Layer (DAL - Data Access Layer)
   ├→ Connection Management
   ├→ Query Building
   └→ Transaction Handling
      ↓
Database Layer
   ├→ JDBC Driver
   ├→ Connection Pool
   └→ Query Executor
```

Each layer is a facade to the layer below!

Real-World Analogy

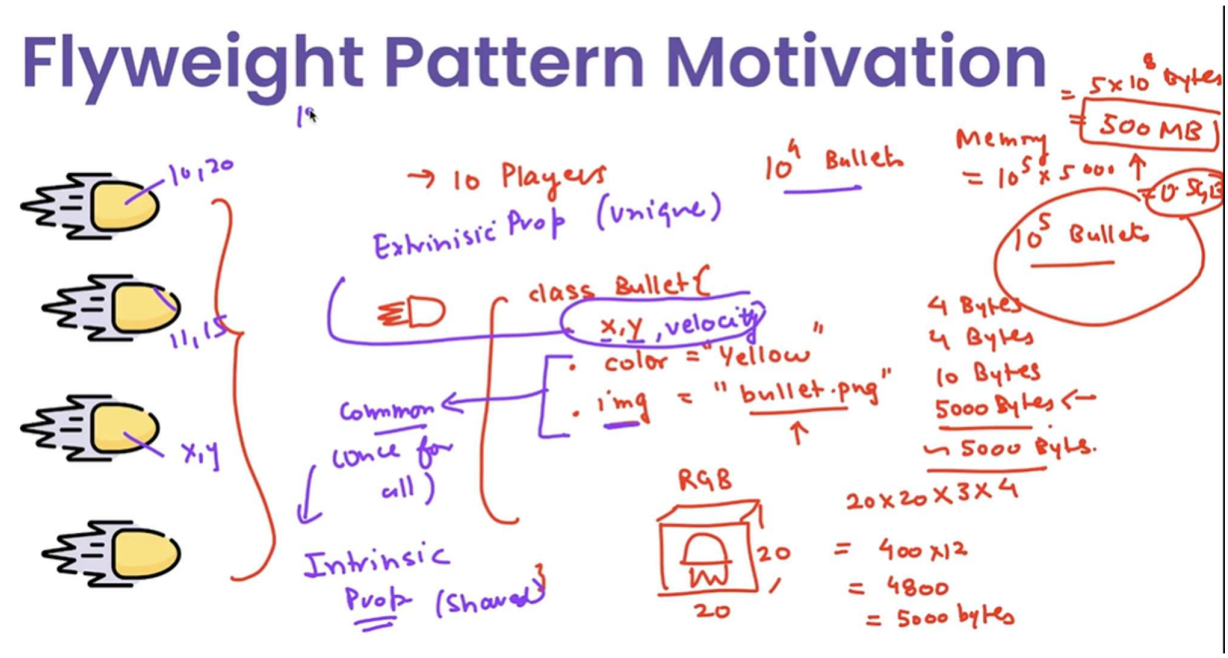
Bank Teller as Facade:

- **Subsystems**: Vault, Accounting, Check Processing, Loan Department
- **Facade**: Bank Teller (accepts requests, coordinates all services)
- **Client**: Customer (doesn't need to know about all departments)
- **Result**: Customer makes one deposit request, teller handles everything!
- API Gateway for multiple backend services
- Database access layer abstracting SQL complexity
- E-commerce workflow combining user, order, and payment services
- Compiler phases (lexer, parser, codegen) abstracted by compiler facade
- Web frameworks simplifying HTTP request/response handling
- Library façades presenting simple interface to complex library
- System initialization, hiding startup complexity
- Business transaction orchestration
- Integration layer combining multiple microservices

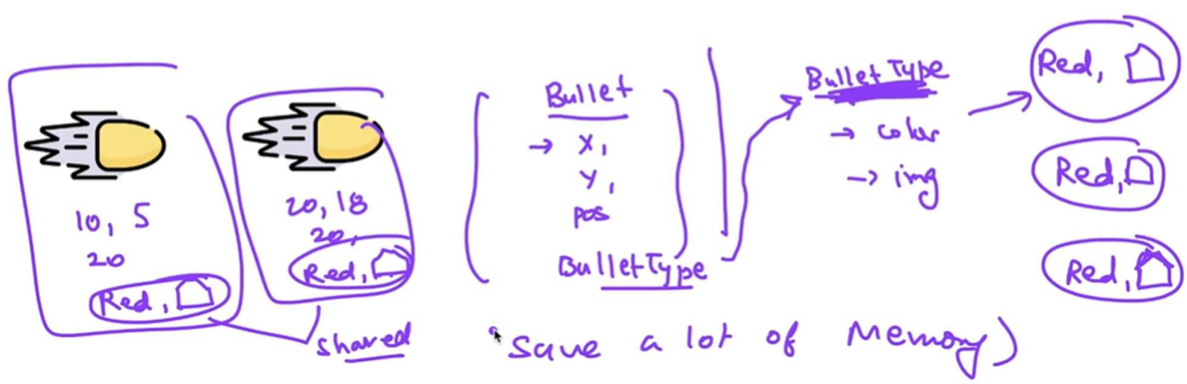
---

6. Flyweight Pattern

In game development, especially in scenarios like shooting games where numerous identical bullets are fired, memory and performance can quickly become an issue if each bullet object stores redundant data. Using the **Flyweight Pattern**, we can reduce memory overhead by sharing intrinsic properties of bullets (like appearance) while maintaining unique extrinsic properties (like position and velocity).



In this refactor, the Bulletclass contains only **extrinsic** data like position and velocity. The **intrinsic** data like color is stored in a BulletTypeclass, which is shared across all Bullet objects. This allows us to manage a large number of bullets efficiently.



Problem: Creating many identical or similar objects in memory-heavy applications can lead to excessive memory consumption.

Solution: By sharing the common state among multiple objects (the **flyweights**), the pattern reduces the memory footprint.

Example Problem**:**

Consider a graphics system rendering a large number of trees in a forest, where each tree shares the same image but may vary by position or size.

The Flyweight Pattern is a structural pattern used to **minimize memory usage** by sharing as much data as possible with similar objects. It separates the **intrinsic state** (shared) from the **extrinsic state** (non-shared).

Purpose: To optimize resource usage in applications that need to handle a large number of similar objects by **reusing** common parts of their data.

When to Use**:**

When you need to create a large number of objects, and they share a lot of common data.

Ideal for memory-constrained applications where the object creation cost is high

Flyweight Pattern Structure

Flyweight: The shared object that stores **intrinsic state** (common data).

Concrete Flyweight: Implements the flyweight interface and shares the intrinsic state.

Flyweight Factory: Manages and creates flyweight objects, ensuring that objects with the same intrinsic state are reused.

Client: Holds and manages the **extrinsic state** (unique information) and delegates behavior to the flyweight.

Flyweight Pattern Example

Scenario: A program that needs to render a large number of trees in a forest.

Intrinsic State: Tree type, shape, texture, and color (shared across trees).

Extrinsic State: Tree position and size (unique per tree).

Without Flyweight: Each tree object would store all properties, leading to **high memory usage**.

With Flyweight: The tree objects share common properties, and only the unique properties (position and size) are stored separately.

Flyweight Design Pattern

The **Flyweight Pattern** is a structural design pattern that uses sharing to support large numbers of fine-grained objects efficiently by sharing common state. It minimizes memory usage by sharing data between multiple objects when those objects have immutable parts. The pattern separates data into intrinsic state (shared) and extrinsic state (unique per object).

Why Use Flyweight Pattern?

- **Memory Optimization**: Drastically reduce memory consumption for many similar objects
- **Performance**: Improve performance in memory-constrained environments
- **Resource Sharing**: Share immutable state among many objects
- **Scalability**: Support thousands/millions of objects efficiently
- **Reduced Object Creation**: Create fewer actual objects
- **Cache Reuse**: Reuse cached objects instead of creating new ones
- **Large-Scale Applications**: Essential for games, simulations, graphics
- **Efficient Collections**: Store large collections efficiently

Key Components

Component	Responsibility
Flyweight Interface	Defines interface for flyweight objects
ConcreteFlyweight	Implements interface, stores intrinsic state
FlyweightFactory	Creates and manages flyweight pool
Client	Works with flyweights, provides extrinsic state

Intrinsic vs Extrinsic State

Aspect	Intrinsic State	Extrinsic State
Definition	Shared, immutable data	Unique, context-dependent data
Storage	Stored in flyweight	Stored in client/context
Shared	Yes (across multiple objects)	No (unique per instance)
Changeable	No (immutable)	Yes (can change)
Memory	Saved by sharing	Not shared
Example (Bullet)	Color (shared)	Position, velocity (unique)

Implementation

1. Flyweight Pattern with Game Bullets

Flyweight Interface & Implementation - BulletType.java:

```
// Flyweight - stores intrinsic (shared) state
public class BulletType {
  private String color;  // Intrinsic state - shared

  public BulletType(String color) {
    this.color = color;
    System.out.println("Creating bullet type with color " + color);
  }

  public String getColor() {
    return color;
  }
}
```

Flyweight Factory - BulletTypeFactory.java:

```
// Factory that manages the flyweight pool
public class BulletTypeFactory {
  // Pool of shared flyweight objects
  private static final HashMap<String, BulletType> bulletTypes = new HashMap<>();

  public static BulletType getBulletType(String color) {
    BulletType type = bulletTypes.get(color);

    // Return existing flyweight if found
    if (type == null) {
      // Create new flyweight only if not found
      type = new BulletType(color);
      bulletTypes.put(color, type);
    }
    return type;
  }
}
```

Context/Client - Bullet.java:

```
// Client that holds extrinsic state and references flyweight
public class Bullet {
  private BulletType type;  // Reference to shared flyweight
  private int x, y;         // Extrinsic state - unique per bullet
  private int velocity;     // Extrinsic state - unique per bullet

  public Bullet(String color, int x, int y, int velocity) {
    // Get shared flyweight from factory
    this.type = BulletTypeFactory.getBulletType(color);
    // Store unique extrinsic state
    this.x = x;
    this.y = y;
    this.velocity = velocity;
```

```
    System.out.println("Creating bullet at (" + x + ", " + y +
                       ") with velocity " + velocity);
  }

  public void display() {
    // Use both intrinsic (from type) and extrinsic state
    System.out.println("Bullet (" + type.getColor() + ") at (" +
                       x + ", " + y + ") moving with velocity " + velocity);
  }
}
```

Key Points:

- BulletType stores only immutable intrinsic state (color)
- Factory maintains pool of BulletType objects
- Bullet stores references to shared flyweight + unique extrinsic state
- Multiple bullets can share same BulletType object
- Significant memory savings with many objects

Flyweight Variants

Approach 1: Simple Flyweight (Shown Above)

- Pros: Simple to understand and implement
- Cons: Manual extrinsic state management
- Best for: Straightforward shared state scenarios

Approach 2: Generic Flyweight Factory

```
public class FlyweightFactory<T> {
  private Map<String, T> pool = new HashMap<>();

  public T getFlyweight(String key, FlyweightCreator<T> creator) {
    if (!pool.containsKey(key)) {
      pool.put(key, creator.create(key));
    }
    return pool.get(key);
  }

  @FunctionalInterface
  public interface FlyweightCreator<T> {
    T create(String key);
  }
}
```

Approach 3: With Reference Counting

```
public class CountedFlyweight {
```

```
  private int referenceCount = 0;

  public void incrementReference() {
    referenceCount++;
  }

  public void decrementReference() {
    referenceCount--;
    if (referenceCount <= 0) {
      // Can be removed from pool
    }
  }
}
```

Approach 4: Thread-Safe Flyweight Factory

```
public class ThreadSafeFlyweightFactory {
  private static final Map<String, BulletType> pool =
    Collections.synchronizedMap(new HashMap<>());

  public static synchronized BulletType getBulletType(String color) {
    return pool.computeIfAbsent(color, k -> new BulletType(k));
  }
}
```

Usage Example

Game.java - Demonstrates flyweight pattern:

```
public class Game {
  public static void main(String[] args) {

    System.out.println("Creating Red Bullets:");
    // 5 red bullets - all reuse same BulletType("red")
    for (int i = 0; i < 5; i++) {
      Bullet bullet = new Bullet("red", i * 10, i * 10, 100);
      bullet.display();
    }

    System.out.println("\n--------------------------------------------\n");

    System.out.println("Creating Blue Bullets:");
    // 5 blue bullets - all reuse same BulletType("blue")
    for (int i = 0; i < 5; i++) {
      Bullet bullet = new Bullet("blue", i * 10, i * 12, 200);
      bullet.display();
    }

    System.out.println("\n--------------------------------------------\n");

    System.out.println("Creating Red Bullets Again:");
    // 5 more red bullets - reuse existing BulletType("red")
```

```
    for (int i = 0; i < 5; i++) {
      Bullet bullet = new Bullet("red", i * 20, i * 20, 150);
      bullet.display();
    }
  }
}
```

Output

```
Creating Red Bullets:
Creating bullet type with color red
Creating bullet at (0, 0) with velocity 100
Bullet (red) at (0, 0) moving with velocity 100
Creating bullet at (10, 10) with velocity 100
Bullet (red) at (10, 10) moving with velocity 100
Creating bullet at (20, 20) with velocity 100
Bullet (red) at (20, 20) moving with velocity 100
... more red bullets without creating new type ...

---------------------------------------------

Creating Blue Bullets:
Creating bullet type with color blue
Creating bullet at (0, 0) with velocity 200
Bullet (blue) at (0, 0) moving with velocity 200
... blue bullets reusing shared BulletType ...

---------------------------------------------

Creating Red Bullets Again:
Creating bullet at (0, 0) with velocity 150
Bullet (red) at (0, 0) moving with velocity 150
... red bullets reusing existing BulletType without creating new one ...
```

Code Flow

1. Client requests flyweight with key (e.g., "red")
2. Factory checks if flyweight already exists in pool
3. If exists, return existing flyweight from pool
4. If not, factory creates new flyweight and stores in pool
5. Client combines flyweight with extrinsic state
6. Multiple clients share same flyweight object
7. Massive memory savings with thousands of objects

Memory Footprint Visualization

Without Flyweight (1000 red bullets):

```
1000 Bullet objects × (color string "red" + x + y + velocity)
```

```
= 1000 copies of "red" string + position + velocity
≈ Very large memory usage
```

With Flyweight (1000 red bullets):

```
1000 Bullet objects × (reference to shared BulletType + x + y + velocity)
+ 1 shared BulletType("red")
= Much smaller memory usage
```

Flyweight vs Related Patterns

Aspect	Flyweight	Object Pool	Cache	Singleton
Purpose	Share intrinsic state	Reuse objects	Store results	Single instance
Objects	Many shared	Reusable	Results only	One
State	Intrinsic/Extrinsic	Stateful	Cached data	Single state
Sharing	Automatic via factory	Manual checkout	Key-based	N/A
Memory	Minimal	Moderate	Selective	Minimal

Real-World Applications

Application	Flyweight	Pool Size Reduction	Example
Game Bullets	BulletType/Color	100x	10K bullets, 100 colors
Text Editor	Character/Font	50x	1M characters, 20 fonts
Browser	Icon/Image	10x	Thousands of pages, icons reused
Forest Scene	Tree Type	100x	1M trees, 10 types
Database Pool	Connection	10x	10 actual, 100 logical
Thread Pool	Worker Thread	20x	20 workers, 400 tasks
String Interning	String Object	100x	Common strings cached
Particle System	Particle Type	50x	100K particles, 20 types

Intrinsic State Examples

Scenario	Shared (Intrinsic)	Unique (Extrinsic)
Game Bullets	Color, damage, type	Position, velocity
Text Editor	Font, size, color	Position in text
Graphics	Texture, material	Transform, scale
UI Buttons	Style, icon, label	Position, state
Database Connections	Driver, credentials	Current query

Real-World Analogy

Movie Theater Seats:

- **Without Flyweight**: Each seat is full object with all properties
- **With Flyweight**: Seats share same type object (color, material)
- **Extrinsic**: Seat location (row, number)
- **Intrinsic**: Seat type (color, material - shared)
- **Result**: Save memory by sharing seat type across 1000s of seats!
- Game development (bullets, particles, enemies, trees in forest)
- Text editors (character formatting, font objects)
- Graphics systems (shapes, colors, textures)
- Web browsers (font caches, icon caches)
- Connection pooling in databases
- Thread pools in concurrent applications
- Particle systems in games/simulations
- Cache management systems
- String interning in Java
- Icon/image caches in UI applications